🌿 KodeQuiz Book

Buku 5 — Coding Prioritas: Python & JavaScript

24 bab · 328 soal dengan pembahasan

Cara memakai buku ini

1. Baca materi tiap bab (±3–5 menit).
2. Kerjakan *Uji Diri* tanpa melihat kunci — di kepala atau di kertas.
3. Cocokkan dengan *Kunci & Pembahasan* di akhir bab. Baca juga kenapa pilihan lain salah.
4. Soal yang salah? Tandai, lalu ulangi besok di aplikasi lewat Ulangan Pintar.

Dibuat 1 November 2026

Daftar Isi

1. Python

1. Dasar: Variabel, Tipe Data & Input/Output
2. Operator: Aritmetika, Perbandingan & Logika
3. String: Indexing, Slicing & Method
4. Percabangan: if / elif / else
5. Perulangan: for, while, range
6. List: Struktur Data Andalan
7. Dict, Tuple & Set
8. Fungsi: def, return, Parameter
9. Error Handling & File
10. OOP: Class, Objek & Inheritance
11. Modul & Ekosistem Python
12. pandas — Excel Versi Kode untuk Analis Data
13. Python untuk Odoo — Model, Field & ORM
14. Latihan Membaca Kode (Gaya Tes Coding)

2. JavaScript

1. Dasar: Variabel, Tipe & Console
2. Operator & Perbandingan
3. Kondisi & Perulangan
4. Fungsi & Arrow Function
5. Array & Method Andalannya
6. String, Objek & Destructuring
7. DOM: Menghidupkan Halaman
8. Asinkron: Promise, Fetch & Storage
9. JavaScript Modern (ES6+) yang Dipakai Setiap Hari
10. Latihan Membaca Kode & Jebakan JavaScript

BAGIAN 1

Python

★ Prioritas #2 — Analisis data (pandas) dan Odoo dibangun dengan Python.

Dasar sampai OOP, pandas, Python untuk Odoo, latihan tes coding

14 bab · 198 soal

PYTHON · BAB 1

Dasar: Variabel, Tipe Data & Input/Output

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Python menyimpan data dalam **variabel** — tanpa perlu menulis tipenya:

```
nama = "Ryan"      # str (teks)
umur = 22          # int (bulat)
ipk = 3.75         # float (desimal)
lulus = True       # bool
```

print() menampilkan output, **input()** menerima ketikan (selalu str, konversi dengan `int()` / `float()`):

```
umur = int(input("Umur: "))
print("Umurmu", umur)
```

Komentar diawali `#`. Nama variabel tidak boleh diawali angka. Nilai kosong ditulis `None`.

🧠 Uji Diri — 15 soal

1. **Fungsi untuk menampilkan output ke layar:**

   A. echo()
   B. print()
   C. console.log()
   D. printf()

2. **Ekstensi file Python adalah...**

   A. .pt
   B. .pyt
   C. .py
   D. .python

3. **Cara membuat komentar satu baris:**

   A. // komentar
   B. <!-- komentar -->
   C. # komentar
   D. /* komentar */

4. **Cara benar mendeklarasikan variabel:**

   A. int x = 5
   B. var x = 5
   C. x = 5
   D. let x = 5

5. **Nama variabel yang TIDAK valid:**

   A. nama_siswa
   B. _total
   C. 2angka
   D. skorAkhir

6. **Tipe data dari 3.14 adalah...**

   A. int
   B. float
   C. str
   D. decimal

7. **Tipe data dari True adalah...**

   A. int
   B. str
   C. bool
   D. float

8. **Fungsi untuk mengetahui tipe sebuah nilai:**

   A. typeof()
   B. type()
   C. dtype()
   D. kind()

9. **input() selalu mengembalikan data bertipe...**

   A. int
   B. str
   C. float
   D. bool

10. **Cara mengubah string "25" menjadi bilangan bulat:**

    A. num("25")
    B. int("25")
    C. parse("25")
    D. str(25)

11. **Hasil print(int(7.9)) adalah...**

    A. 8
    B. 7.9
    C. 7
    D. error

12. **Hasil print(type("5")) menunjukkan...**

    A. int
    B. str
    C. float
    D. number

13. **Nilai kosong/tidak ada di Python ditulis ____**

    Jawab: _____

14. **Fungsi untuk menerima ketikan pengguna: ____()**

    Jawab: _____

15. **Fungsi konversi ke bilangan desimal: ____("3.5")**

    Jawab: _____

Kunci & Pembahasan · Python Bab 1

← Kembali ke soal · Awal bab

1. **B. print()** — print() adalah fungsi bawaan Python untuk output.
2. **C. .py** — File Python berekstensi .py
3. **C. # komentar** — Python memakai tanda pagar (#).
4. **C. x = 5** — Python tidak butuh kata kunci tipe: cukup x = 5.
5. **C. 2angka** — Nama variabel tidak boleh diawali angka.
6. **B. float** — Bilangan desimal bertipe float.
7. **C. bool** — True dan False bertipe bool.
8. **B. type()** — type(x) mengembalikan tipe data x.
9. **B. str** — Hasil input() selalu string; konversi dengan int()/float() bila perlu.
10. **B. int("25")** — int() mengonversi ke integer.
11. **C. 7** — int() memotong desimal (bukan membulatkan): 7.
12. **B. str** — Angka dalam tanda kutip adalah string.
13. **none** — None mewakili ketiadaan nilai.
14. **input** — input() membaca teks dari keyboard.
15. **float** — float() mengonversi ke bilangan desimal.

Operator: Aritmetika, Perbandingan & Logika

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Aritmetika: `+ - * /`, lalu tiga yang khas Python:

```
2 ** 3   # pangkat  = 8
10 // 3  # bagi bulat = 3
10 % 3   # sisa bagi  = 1
```

Perbandingan menghasilkan bool: `== != > < >= <=`. Ingat: `=` memberi nilai, `==` membandingkan.

Logika memakai kata: `and` (semua benar), `or` (salah satu benar), `not` (membalik). Singkatan: `x += 3` sama dengan `x = x + 3`.

🧠 Uji Diri — 13 soal

1. **Hasil print(2 ** 3):**

   A. 6
   B. 8
   C. 9
   D. 23

2. **Hasil print(10 // 3):**

   A. 3.33
   B. 3
   C. 4
   D. 1

3. **Hasil print(10 % 3):**

   A. 3
   B. 0
   C. 1
   D. 3.3

4. **Hasil print(7 / 2):**

   A. 3
   B. 3.5
   C. 4
   D. 3,5

5. **Perbedaan = dan == :**

   A. sama saja
   B. = penugasan, == perbandingan
   C. == penugasan, = perbandingan
   D. keduanya perbandingan

6. **x = 5; x += 3; nilai x sekarang...**

   A. 5
   B. 3
   C. 8
   D. 53

7. **Hasil print(5 > 3 and 2 > 4):**

   A. True
   B. False
   C. error
   D. None

8. **Hasil print(5 > 3 or 2 > 4):**

    A. True
    B. False
    C. error
    D. None

9. **Hasil print(not True):**

    A. True
    B. False
    C. 0
    D. error

10. **Hasil print(7 == "7"):**

    A. True
    B. False
    C. error
    D. 7

11. **Operator untuk mengecek keanggotaan ("a" ada di list?):**

    A. has
    B. in
    C. of
    D. inside

12. **Operator pangkat di Python ditulis ____**

    Jawab: _____

13. **Operator logika "dan" di Python: ____**

    Jawab: _____

Kunci & Pembahasan · Python Bab 2

← Kembali ke soal · Awal bab

1. **B. 8** — ** adalah pangkat: $2^3 = 8$.
2. **B. 3** — // pembagian bulat (buang desimal).
3. **C. 1** — % modulus = sisa bagi. 10÷3 sisa 1.
4. **B. 3.5** — Operator / selalu menghasilkan float.
5. **B. = penugasan, == perbandingan** — = memberi nilai; == membandingkan.
6. **C. 8** — x += 3 sama dengan x = x + 3.
7. **B. False** — and butuh keduanya benar; 2>4 salah → False.
8. **A. True** — or cukup salah satu benar.
9. **B. False** — not membalik nilai boolean.
10. **B. False** — int dan str berbeda tipe → False.
11. **B. in** — "a" in daftar → True jika ada.
12. ****** — 2 ** 4 = 16.
13. **and** — Python memakai kata: and, or, not.

String: Indexing, Slicing & Method

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

String adalah deretan karakter berindeks mulai **0**; indeks negatif dihitung dari belakang:

```
s = "informatika"
s[0]    # "i"
s[-1]   # "a"
s[0:4]  # "info"  (slicing: awal:akhir, akhir tak termasuk)
```

Method penting: `upper()` `lower()` `strip()` `replace(a,b)` `split(",")` `count(x)` dan `len(s)` untuk panjang. Menyisipkan variabel paling rapi dengan **f-string**:

```
print(f"Halo {nama}, umurmu {umur}")
```

🧠 Uji Diri — 15 soal

1. **Hasil print("a" + "b"):**

   A. ab
   B. a+b
   C. error
   D. a b

2. **Hasil print("ha" * 3):**

   A. hahaha
   B. ha3
   C. error
   D. hhaaa

3. **Hasil print(len("Python")):**

   A. 5
   B. 6
   C. 7
   D. error

4. **s = "koding"; print(s[0]) menghasilkan...**

   A. k
   B. o
   C. g
   D. koding

5. **s = "informatika"; print(s[0:4]):**

   A. info
   B. infor
   C. nfor
   D. tika

6. **print("python"[-1]) menghasilkan...**

   A. p
   B. n
   C. o
   D. error

7. **Method mengubah semua huruf jadi kecil:**

   A. small()
   B. lower()
   C. down()
   D. mini()

8. **Method menghapus spasi di awal & akhir string:**

   A. clean()
   B. trim()
   C. strip()
   D. cut()

9. **nama="Ryan"; cara benar menyisipkan ke string:**

   A. print(f"Halo {nama}")
   B. print("Halo {nama}")
   C. print("Halo" nama)
   D. print($"Halo {nama}")

10. **Method mengganti kata: s.____("a","b")**

    A. swap
    B. change
    C. replace
    D. edit

11. **"a,b,c".split(",") menghasilkan...**

    A. "abc"
    B. ["a","b","c"]
    C. ("a","b","c")
    D. error

12. **"-".join(["a","b"]) menghasilkan...**

    A. ab
    B. a-b
    C. [a-b]
    D. error

13. **"python".count("o") menghasilkan...**

    A. 0
    B. 1
    C. 2
    D. o

14. **Method huruf kapital semua: teks.____()**

    Jawab: _____

15. **"belajar".startswith("bel") mengembalikan ____**

    Jawab: _____

Kunci & Pembahasan · Python Bab 3

← Kembali ke soal · Awal bab

1. **A. ab** — + pada string = penggabungan.
2. **A. hahaha** — String × angka = pengulangan.
3. **B. 6** — "Python" = 6 karakter.
4. **A. k** — Indeks dimulai dari 0.
5. **A. info** — Slicing [0:4] = indeks 0–3.
6. **B. n** — Indeks -1 = karakter terakhir.
7. **B. lower()** — "ABC".lower() → "abc".
8. **C. strip()** — Python memakai strip(), bukan trim().
9. **A. print(f"Halo {nama}")** — f-string: awali dengan f, variabel dalam { }.
10. **C. replace** — replace(lama, baru) mengganti substring.
11. **B. ["a","b","c"]** — split memecah string menjadi list.
12. **B. a-b** — join menggabungkan list dengan pemisah.
13. **B. 1** — count menghitung kemunculan substring.
14. **upper** — upper() mengubah jadi huruf besar.
15. **true** — startswith mengecek awalan string → True.

Percabangan: if / elif / else

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Python menentukan blok kode dengan **indentasi** (4 spasi), bukan kurung kurawal:

```
if nilai >= 80:
    print("A")
elif nilai >= 70:
    print("B")
else:
    print("C")
```

Nilai yang dianggap **False** (falsy): `0`, `""`, `[]`, `None`. Ternary satu baris: `status = "Lulus" if n >= 75 else "Gagal"`. Untuk None gunakan `is not None`.

🧠 Uji Diri — 12 soal

1. **Penulisan if yang benar:**

   A. if x > 5 then:
   B. if (x > 5) { }
   C. if x > 5:
   D. when x > 5:

2. **Blok kode Python dibedakan dengan…**

   A. kurung kurawal
   B. titik koma
   C. indentasi
   D. kata end

3. **Kata kunci untuk kondisi alternatif kedua:**

   A. elseif
   B. elif
   C. else if
   D. ef

4. **x=10. Output dari: if x>5: print("A") else: print("B")**

   A. A
   B. B
   C. AB
   D. error

5. **Hasil bool("") adalah...**

   A. True
   B. False
   C. error
   D. None

6. **Nilai yang dianggap False:**

   A. 1
   B. "0"
   C. 0
   D. "False"

7. **nilai=75. if nilai>=75: print("Lulus"). Outputnya...**

   A. Lulus
   B. tidak ada
   C. error
   D. False

8. **Ternary yang benar:**

   A. x = "A" if n>5 else "B"
   B. x = if n>5 "A" else "B"
   C. x = n>5 ? "A" : "B"
   D. x = (n>5) then A

9. **if umur>=17 and punya_ktp: ... berjalan jika...**

   A. salah satu benar
   B. keduanya benar
   C. keduanya salah
   D. umur saja

10. **Pengecekan x bukan None yang paling dianjurkan:**

    A. if x != None:
    B. if x is not None:
    C. if not x == None:
    D. if x <> None:

11. **Kata kunci cabang terakhir jika semua kondisi gagal: ____:**

    Jawab: _____

12. **Operator "tidak sama dengan": x ____ y**

    Jawab: _____

Kunci & Pembahasan · Python Bab 4

← Kembali ke soal · Awal bab

1. **C. if x > 5:** — if kondisi: lalu blok menjorok (indentasi).
2. **C. indentasi** — Indentasi (biasanya 4 spasi) menentukan blok.
3. **B. elif** — Python memakai elif.
4. **A. A** — 10 > 5 benar → cabang if dijalankan.
5. **B. False** — String kosong bernilai False (falsy).
6. **C. 0** — 0, "", [], None → falsy. "0" adalah string berisi → truthy.
7. **A. Lulus** — >= berarti lebih besar ATAU sama dengan.
8. **A. x = "A" if n>5 else "B"** — Bentuk: nilai_jika_benar if kondisi else nilai_jika_salah.
9. **B. keduanya benar** — and menuntut semua kondisi benar.
10. **B. if x is not None:** — Gunakan is/is not untuk None.
11. **else** — else menangani sisa kasus.
12. **!=** — != berarti tidak sama dengan.

Perulangan: for, while, range

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

for mengulang tiap elemen; **range(awal, akhir, langkah)** membuat deret (akhir tak termasuk):

```
for i in range(3):       # 0 1 2
    print(i)
for i in range(2, 10, 3): # 2 5 8
    ...
```

while mengulang selama kondisi True — pastikan kondisinya bisa berubah agar tidak infinite loop. `break` menghentikan loop, `continue` melompati satu iterasi. Bonus mahir: `enumerate()` memberi indeks+nilai, `zip()` memasangkan dua list.

🧠 Uji Diri — 15 soal

1. **Output: for i in range(3): print(i)**

   A. 1 2 3
   B. 0 1 2
   C. 0 1 2 3
   D. 1 2

2. **range(1, 5) menghasilkan...**

   A. 1,2,3,4,5
   B. 1,2,3,4
   C. 0,1,2,3,4
   D. 2,3,4,5

3. **Kata kunci menghentikan loop lebih awal:**

   A. stop
   B. exit
   C. break
   D. end

4. **Kata kunci melewati satu iterasi:**

   A. skip
   B. pass
   C. continue
   D. next

5. **range(2, 10, 3) menghasilkan...**

   A. 2,5,8
   B. 2,4,6,8
   C. 3,6,9
   D. 2,5,8,11

6. **while berjalan selama...**

   A. kondisi False
   B. kondisi True
   C. 10 kali
   D. selamanya selalu

7. **Kode yang menyebabkan infinite loop:**

   A. while True: (tanpa break)
   B. for i in range(3):
   C. while x < 5: x += 1
   D. for x in [1,2]:

8. **total=0; for i in range(1,4): total+=i. Nilai total:**

   A. 3
   B. 6
   C. 10
   D. 4

9. **Cara mengulangi setiap huruf dalam string s:**

   A. for h in s:
   B. for h of s:
   C. foreach h in s:
   D. loop s:

10. **Fungsi untuk mendapat indeks sekaligus nilai saat loop:**

    A. index()
    B. enumerate()
    C. count()
    D. pairs()

11. **Output: for i in range(5): if i==3: break; print(i) (print di dalam loop)**

    A. 0 1 2
    B. 0 1 2 3
    C. 3 4
    D. 0 1 2 3 4

12. **Loop bersarang: for i in range(2): for j in range(3): ... dieksekusi berapa kali?**

    A. 5
    B. 6
    C. 2
    D. 3

13. **zip([1,2],["a","b"]) memasangkan menjadi...**

    A. (1,"a"),(2,"b")
    B. [1,2,"a","b"]
    C. {1:2}
    D. error

14. **Fungsi pembuat deret angka untuk loop: for i in ____(10):**

    Jawab: _____

15. **Loop yang mengulang selama kondisi benar: ____ x < 10:**

    Jawab: _____

Kunci & Pembahasan · Python Bab 5

← Kembali ke soal · Awal bab

1. **B. 0 1 2** — range(3) = 0,1,2 (batas atas tak termasuk).
2. **B. 1,2,3,4** — Mulai 1, berhenti sebelum 5.
3. **C. break** — break keluar dari loop.
4. **C. continue** — continue lompat ke iterasi berikutnya.
5. **A. 2,5,8** — Mulai 2, langkah 3: 2,5,8.
6. **B. kondisi True** — while mengulang selama kondisinya True.
7. **A. while True: (tanpa break)** — while True tanpa break tak pernah berhenti.
8. **B. 6** — 1+2+3 = 6.
9. **A. for h in s:** — String bisa langsung diiterasi dengan for...in.
10. **B. enumerate()** — for i, v in enumerate(daftar):
11. **A. 0 1 2** — Saat i=3 loop berhenti sebelum print.
12. **B. 6** — 2 × 3 = 6 kali.
13. **A. (1,"a"),(2,"b")** — zip menggabungkan elemen sejajar dari beberapa iterable.
14. **range** — range menghasilkan deret angka.
15. **while** — while mengulang berdasarkan kondisi.

List: Struktur Data Andalan

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

List: koleksi terurut yang bisa diubah (mutable), ditulis `[ ]`:

```
a = [3, 1, 2]
a.append(9)   # tambah di akhir
a.remove(1)   # hapus nilai 1
a.sort()      # urutkan
len(a), sum(a), max(a), min(a)
```

Indeks & slicing sama seperti string: `a[0]`, `a[-1]`, `a[1:3]`. **Hati-hati:** `b = a` menunjuk list yang sama — pakai `a.copy()`. Cara jenius membuat list: **list comprehension**:

```
genap = [x for x in range(10) if x % 2 == 0]
```

🧠 Uji Diri — 15 soal

1. **Cara membuat list:**

   A. a = (1,2,3)
   B. a = {1,2,3}
   C. a = [1,2,3]
   D. a = <1,2,3>

2. **angka=[5,7,9]; angka[0] adalah...**

   A. 5
   B. 7
   C. 9
   D. error

3. **Method menambah elemen di akhir list:**

   A. add()
   B. push()
   C. insert()
   D. append()

4. **Fungsi menghitung jumlah elemen list:**

   A. size()
   B. count()
   C. len()
   D. length()

5. **a=[1,2,3]; a[1]=9; a menjadi...**

   A. [9,2,3]
   B. [1,9,3]
   C. [1,2,9]
   D. error

6. **Method menghapus elemen berdasarkan NILAI:**

   A. remove()
   B. pop()
   C. delete()
   D. del()

7. **a=[3,1,2]; a.sort(); a menjadi...**

   A. [3,1,2]
   B. [1,2,3]
   C. [3,2,1]
   D. [2,1,3]

8. **a=[1,2,3]; a[-1] adalah...**

   A. 1
   B. 2
   C. 3
   D. error

9. **Hasil [1,2] + [3]:**

   A. [1,2,3]
   B. [6]
   C. [[1,2],3]
   D. error

10. **Fungsi menjumlahkan semua elemen [1,2,3]:**

    A. total()
    B. sum()
    C. add()
    D. plus()

11. **Hasil [x*2 for x in range(3)]:**

    A. [0,2,4]
    B. [2,4,6]
    C. [0,1,2]
    D. error

12. **[x for x in range(6) if x%2==0] menghasilkan...**

    A. [0,2,4]
    B. [1,3,5]
    C. [2,4,6]
    D. [0,1,2]

13. **a=[1,2,3]; b=a; b.append(4). Isi a sekarang...**

    A. [1,2,3]
    B. [1,2,3,4]
    C. error
    D. [4]

14. **Fungsi mencari nilai terbesar list: ____([4,9,2])**

    Jawab: _____

15. **Method membalik urutan list secara langsung: a.____()**

    Jawab: _____

Kunci & Pembahasan · Python Bab 6

← Kembali ke soal · Awal bab

1. **C. a = [1,2,3]** — List memakai kurung siku [ ].
2. **A. 5** — Indeks 0 = elemen pertama.
3. **D. append()** — append(x) menambah di akhir.
4. **C. len()** — len(daftar).
5. **B. [1,9,3]** — List mutable — indeks 1 diganti.
6. **A. remove()** — remove(nilai); pop(indeks) berdasarkan posisi.
7. **B. [1,2,3]** — sort() mengurutkan naik secara langsung (in-place).
8. **C. 3** — -1 = elemen terakhir.
9. **A. [1,2,3]** — + menggabungkan dua list.
10. **B. sum()** — sum([1,2,3]) = 6.
11. **A. [0,2,4]** — List comprehension: 0,2,4.
12. **A. [0,2,4]** — Comprehension dengan filter genap.
13. **B. [1,2,3,4]** — b=a menunjuk objek yang sama — gunakan a.copy() agar terpisah.
14. **max** — max() nilai terbesar; min() terkecil.
15. **reverse** — reverse() membalik list in-place.

Dict, Tuple & Set

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Dict menyimpan pasangan kunci→nilai:

```
mhs = {"nama": "Ryan", "nim": "220103091"}
mhs["nama"]          # akses
mhs["kota"] = "Solo" # tambah
mhs.get("ipk", 0)    # aman, ada default
for k, v in mhs.items(): ...
```

Tuple `(1, 2)`: terurut tapi *tidak bisa diubah* — cocok untuk data tetap. **Set** `{1, 2, 3}`: tanpa urutan, otomatis tanpa duplikat; operasi: `&` irisan, `|` gabungan.

🧠 Uji Diri — 15 soal

1. **Struktur data pasangan kunci→nilai:**

   A. list
   B. tuple
   C. dict
   D. set

2. **d={"nama":"Ryan"}; cara mengambil nilainya:**

   A. d.nama
   B. d→nama
   C. d["nama"]
   D. d(nama)

3. **Struktur terurut yang TIDAK bisa diubah:**

   A. list
   B. dict
   C. tuple
   D. set

4. **Struktur tanpa urutan dan otomatis tanpa duplikat:**

   A. list
   B. set
   C. tuple
   D. dict

5. **Hasil set([1,2,2,3,3]):**

   A. [1,2,3]
   B. {1,2,3}
   C. (1,2,3)
   D. {1,2,2,3,3}

6. **Cara menambah pasangan baru ke dict d:**

   A. d.add("kota","Solo")
   B. d["kota"]="Solo"
   C. d.append("kota")
   D. d+"kota"

7. **Method dict mengambil semua kunci:**

   A. keys()
   B. all()
   C. index()
   D. names()

8. **d.get("x", 0) berguna karena...**

   A. lebih cepat
   B. tidak error jika kunci tak ada (kembalikan default)
   C. menghapus x
   D. mengurutkan

9. **t=(1,2,3); t[0]=9 menghasilkan...**

   A. (9,2,3)
   B. error
   C. (1,9,3)
   D. 9

10. **Cara loop kunci dan nilai dict sekaligus:**

    A. for k,v in d.items():
    B. for k,v in d:
    C. for d.pairs():
    D. loop d:

11. **len({"a":1,"b":2}) menghasilkan...**

    A. 1
    B. 2
    C. 3
    D. 4

12. **Menghapus kunci "x" dari dict d:**

    A. d.remove("x")
    B. del d["x"]
    C. d.delete("x")
    D. d - "x"

13. **{1,2} & {2,3} (irisan set) menghasilkan...**

    A. {1,2,3}
    B. {2}
    C. {1,3}
    D. error

14. **"kota" ____ d mengecek apakah kunci ada di dict**

    Jawab: _____

15. **Fungsi mengubah list menjadi tuple: ____([1,2])**

    Jawab: _____

Kunci & Pembahasan · Python Bab 7

← Kembali ke soal · Awal bab

1. **C. dict** — Dictionary: {"kunci": nilai}.
2. **C. d["nama"]** — Akses dict dengan kunci dalam kurung siku.
3. **C. tuple** — Tuple immutable, ditulis dengan ( ).
4. **B. set** — Set menyimpan elemen unik.
5. **B. {1,2,3}** — set menghapus duplikat.
6. **B. d["kota"]="Solo"** — Assign langsung: d["kunci"] = nilai.
7. **A. keys()** — d.keys(); nilai: d.values(); pasangan: d.items().
8. **B. tidak error jika kunci tak ada (kembalikan default)** — get mengembalikan default alih-alih KeyError.
9. **B. error** — Tuple tidak bisa diubah → TypeError.
10. **A. for k,v in d.items():** — items() menghasilkan pasangan (kunci, nilai).
11. **B. 2** — len dict = jumlah pasangan.
12. **B. del d["x"]** — del d["kunci"] atau d.pop("kunci").
13. **B. {2}** — & = irisan; | = gabungan; - = selisih.
14. **in** — Operator in mengecek keberadaan kunci.
15. **tuple** — tuple() mengonversi iterable ke tuple.

Fungsi: def, return, Parameter

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Fungsi membungkus kode agar bisa dipakai ulang:

```
def luas(p, l=1):        # l punya nilai default
    return p * l

print(luas(4, 5))        # 20
print(luas(p=4))         # keyword argument
```

Tanpa `return`, fungsi mengembalikan `None`. Variabel di dalam fungsi bersifat **lokal**. Bisa mengembalikan banyak nilai: `return a, b`. Fungsi mini satu baris: `kali2 = lambda x: x * 2`. `*args` menampung argumen bebas.

🧠 Uji Diri — 15 soal

1. **Kata kunci membuat fungsi:**

   A. function
   B. func
   C. def
   D. fn

2. **Kata kunci mengembalikan nilai dari fungsi:**

   A. give
   B. return
   C. back
   D. out

3. **def sapa(nama): ... . nama disebut...**

   A. argumen
   B. parameter
   C. variabel global
   D. atribut

4. **def f(x): return x*2. Hasil print(f(4)):**

   A. 4
   B. 8
   C. x*2
   D. error

5. **Fungsi tanpa return mengembalikan...**

   A. 0
   B. ""
   C. None
   D. error

6. **def f(nama="Tamu"): ... . "Tamu" adalah...**

   A. parameter wajib
   B. nilai default
   C. konstanta
   D. global

7. **Memanggil f(b=2, a=1) memakai...**

   A. positional argument
   B. keyword argument
   C. default
   D. lambda

8. **Variabel yang dibuat di dalam fungsi bersifat...**

   A. global
   B. lokal
   C. statis
   D. permanen

9. **lambda x: x+1 adalah...**

   A. fungsi anonim satu ekspresi
   B. loop
   C. class
   D. error

10. ***args pada parameter menampung...**

    A. argumen posisi bebas (tuple)
    B. keyword bebas (dict)
    C. satu argumen
    D. file

11. **Fungsi bisa mengembalikan lebih dari satu nilai lewat...**

    A. tidak bisa
    B. return a, b (tuple)
    C. dua return
    D. global

12. **Fungsi yang memanggil dirinya sendiri disebut...**

    A. loop
    B. rekursif
    C. lambda
    D. callback

13. **Teks dokumentasi di baris pertama fungsi disebut...**

    A. komentar
    B. docstring
    C. header
    D. meta

14. **Kata kunci mengakses variabel global di dalam fungsi: ____ skor**

    Jawab: _____

15. **Fungsi anonim dibuat dengan kata kunci ____**

    Jawab: _____

Kunci & Pembahasan · Python Bab 8

← Kembali ke soal · Awal bab

1. **C. def** — def nama_fungsi():
2. **B. return** — return mengirim hasil ke pemanggil.
3. **B. parameter** — Parameter di definisi; argumen saat pemanggilan.
4. **B. 8** — f(4) mengembalikan 8.
5. **C. None** — Tanpa return, hasilnya None.
6. **B. nilai default** — Nilai default dipakai jika argumen tidak diberikan.
7. **B. keyword argument** — Keyword argument menyebut nama parameternya.
8. **B. lokal** — Variabel lokal hanya hidup di dalam fungsinya.
9. **A. fungsi anonim satu ekspresi** — lambda membuat fungsi kecil tanpa nama.
10. **A. argumen posisi bebas (tuple)** — *args → tuple; **kwargs → dict keyword.
11. **B. return a, b (tuple)** — return a, b mengembalikan tuple yang bisa di-unpack.
12. **B. rekursif** — Rekursi butuh kondisi berhenti (base case).
13. **B. docstring** — Docstring ditulis dengan """ """ dan bisa dibaca help().

14. **global** — global memberi akses tulis ke variabel global.
15. **lambda** — lambda x: x*2

Error Handling & File

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Tangani error agar program tidak crash:

```
try:
    n = int(input("Angka: "))
except ValueError:
    print("Itu bukan angka!")
finally:
    print("Selalu jalan")
```

Error umum: **ValueError** (nilai salah), **NameError** (variabel tak dikenal), **IndexError**, **ZeroDivisionError**. Membaca/menulis file paling aman dengan `with` (auto-close):

```
with open("data.txt", "r") as f:   # "w" tulis, "a" tambah
    isi = f.read()
```

🧠 Uji Diri — 12 soal

1. **Blok penanganan error di Python:**

   A. try/catch
   B. try/except
   C. do/rescue
   D. if/error

2. **int("abc") memunculkan error jenis...**

   A. TypeError
   B. ValueError
   C. KeyError
   D. NameError

3. **Mengakses variabel yang belum dibuat menimbulkan...**

   A. ValueError
   B. NameError
   C. IndexError
   D. KeyError

4. **a=[1,2]; a[5] menimbulkan...**

   A. IndexError
   B. KeyError
   C. ValueError
   D. None

5. **Blok yang SELALU dijalankan (ada error ataupun tidak):**

   A. else
   B. finally
   C. always
   D. end

6. **Cara membuka file yang otomatis menutup:**

   A. open(f).close()
   B. with open(f) as x:
   C. read(f)
   D. auto open(f)

7. **Mode "w" pada open() berarti...**

   A. baca
   B. tulis (timpa isi lama)
   C. tambah di akhir
   D. baca-tulis

8. **Mode untuk menambah teks di akhir file tanpa menghapus:**

   A. "r"
   B. "w"
   C. "a"
   D. "x"

9. **Melempar error secara manual memakai...**

   A. throw
   B. raise
   C. error()
   D. panic

10. **f.read() vs f.readlines():**

    A. sama

    B. read=1 string utuh, readlines=list per baris

    C. readlines lebih cepat

    D. read per baris

11. **try: x=1/0 → except ____Error: menangkap pembagian nol**

    Jawab: _____

12. **Kata kunci placeholder blok kosong agar tidak error: ____**

    Jawab: _____

Kunci & Pembahasan · Python Bab 9

← Kembali ke soal · Awal bab

1. **B. try/except** — Python memakai try...except.
2. **B. ValueError** — Nilai tidak valid untuk konversi → ValueError.
3. **B. NameError** — NameError: variabel tidak dikenal.
4. **A. IndexError** — Indeks di luar jangkauan → IndexError.
5. **B. finally** — finally cocok untuk bersih-bersih (menutup file, dll).
6. **B. with open(f) as x:** — with ... as menutup file otomatis.
7. **B. tulis (timpa isi lama)** — "r" baca, "w" tulis-timpa, "a" append.
8. **C. "a"** — "a" = append.
9. **B. raise** — raise ValueError("pesan").
10. **B. read=1 string utuh, readlines=list per baris** — read() seluruh isi; readlines() list baris.
11. **zerodivision** — 1/0 memunculkan ZeroDivisionError.
12. **pass** — pass = tidak melakukan apa-apa.

OOP: Class, Objek & Inheritance

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Class = cetakan; objek = hasil cetakannya:

```
class Mahasiswa:
    def __init__(self, nama):   # konstruktor
        self.nama = nama        # atribut
    def sapa(self):
        return f"Halo, {self.nama}"

m = Mahasiswa("Ryan")   # objek
print(m.sapa())
```

`self` merujuk objek yang sedang dipakai. **Inheritance**: `class Asisten(Mahasiswa):` mewarisi semuanya; `super().__init__()` memanggil konstruktor induk; method bisa di-*override*. Awalan `__` menandakan atribut privat (enkapsulasi).

🧠 Uji Diri — 13 soal

1. **Kata kunci membuat class:**

   A. object
   B. class
   C. struct
   D. type

2. **Method __init__ berperan sebagai...**

   A. destructor
   B. konstruktor (inisialisasi objek)
   C. import
   D. loop

3. **Parameter self merujuk pada...**

   A. class
   B. objek/instance yang dipakai
   C. modul
   D. global

4. **m = Mahasiswa(). m disebut...**

   A. class
   B. objek/instance
   C. method
   D. atribut

5. **self.nama = nama di dalam __init__ membuat...**

   A. variabel lokal
   B. atribut objek
   C. fungsi
   D. class baru

6. **class Dosen(Pegawai): berarti...**

   A. Dosen berisi Pegawai
   B. Dosen mewarisi Pegawai (inheritance)
   C. keduanya sama
   D. error

7. **Fungsi memanggil konstruktor class induk:**

   A. parent()
   B. super().__init__()
   C. base()
   D. init.parent()

8. **Method dengan nama sama tapi perilaku beda di class anak disebut...**

   A. overloading
   B. overriding
   C. inheritance
   D. encapsulation

9. **Atribut dengan awalan __ (dua underscore) bersifat...**

   A. publik
   B. privat (name mangling)
   C. statis
   D. global

10. **isinstance(x, Mahasiswa) mengecek...**

    A. x class atau bukan
    B. apakah x objek dari Mahasiswa
    C. jumlah objek
    D. nama x

11. **Prinsip menyembunyikan detail internal class disebut...**

    A. inheritance
    B. polymorphism
    C. enkapsulasi
    D. abstraksi total

12. **Parameter pertama setiap method instance: ____**

    Jawab: _____

13. **Method spesial dipanggil print(objek): __ ____ __**

    Jawab: _____

Kunci & Pembahasan · Python Bab 10

← Kembali ke soal · Awal bab

1. **B. class** — class Mahasiswa:
2. **B. konstruktor (inisialisasi objek)** — __init__ dipanggil otomatis saat objek dibuat.
3. **B. objek/instance yang dipakai** — self = instance yang sedang memanggil method.
4. **B. objek/instance** — Objek adalah hasil 'cetakan' dari class.
5. **B. atribut objek** — Atribut melekat pada tiap objek.
6. **B. Dosen mewarisi Pegawai (inheritance)** — Inheritance: Dosen mendapat atribut & method Pegawai.
7. **B. super().__init__()** — super() mengakses class induk.
8. **B. overriding** — Override: class anak menimpa method induk.
9. **B. privat (name mangling)** — __x menandakan privat — bagian dari enkapsulasi.
10. **B. apakah x objek dari Mahasiswa** — isinstance mengecek tipe/kelas objek.
11. **C. enkapsulasi** — Enkapsulasi membungkus data + method dan membatasi akses.
12. **self** — self selalu parameter pertama method.

13. **str** — __str__ menentukan representasi teks objek.

Modul & Ekosistem Python

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Modul = kumpulan kode siap pakai:

```
import math
math.sqrt(16)            # 4.0

from random import randint
randint(1, 6)            # dadu: 1-6

import numpy as np       # alias
```

Paket eksternal dipasang lewat terminal: `pip install nama_paket`. Idiom penting di file yang bisa di-import: `if __name__ == "__main__":` — blok itu hanya jalan saat file dieksekusi langsung.

🧠 Uji Diri — 10 soal

1. **Kata kunci memuat modul:**

   A. include
   B. import
   C. using
   D. load

2. **Modul standar untuk angka acak:**

   A. math
   B. random
   C. numpy
   D. rand

3. **from math import sqrt memungkinkan...**

   A. sqrt() langsung tanpa math.
   B. semua fungsi math
   C. error
   D. import ganda

4. **import numpy as np. "np" disebut...**

   A. variabel
   B. alias
   C. fungsi
   D. class

5. **math.sqrt(16) menghasilkan...**

   A. 4.0
   B. 8
   C. 256
   D. error

6. **Perintah memasang paket eksternal:**

   A. python get
   B. pip install nama
   C. import install
   D. apt python

7. **random.randint(1, 6) menghasilkan...**

   A. 1-5
   B. 1-6 (keduanya termasuk)
   C. 0-6
   D. 0-5

8. **if __name__ == "__main__": berguna untuk...**

   A. import cepat
   B. kode hanya jalan saat file dieksekusi langsung
   C. loop utama
   D. error handling

9. **Modul standar berisi sqrt, pi, floor: import ____**

   Jawab: _____

10. **Fungsi mengacak urutan list: random.____(daftar)**

    Jawab: _____

Kunci & Pembahasan · Python Bab 11

← Kembali ke soal · Awal bab

1. **B. import** — import math
2. **B. random** — random.randint(a,b) contohnya.
3. **A. sqrt() langsung tanpa math.** — from...import mengambil fungsi tertentu saja.
4. **B. alias** — as memberi nama singkat (alias).
5. **A. 4.0** — sqrt = akar kuadrat, hasil float.
6. **B. pip install nama** — pip adalah package manager Python.
7. **B. 1-6 (keduanya termasuk)** — randint inklusif di kedua ujung — cocok untuk dadu.
8. **B. kode hanya jalan saat file dieksekusi langsung** — Blok ini tidak ikut jalan saat file di-import modul lain.
9. **math** — math berisi fungsi matematika.
10. **shuffle** — random.shuffle mengacak list di tempat.

pandas — Excel Versi Kode untuk Analis Data

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

pandas adalah library Python nomor satu untuk analisis data. Kalau kamu sudah paham Excel dan SQL, pandas tinggal belajar "cara menulisnya".

```
import pandas as pd
df = pd.read_csv("penjualan.csv")   # atau
pd.read_excel("file.xlsx")

df.head()           # 5 baris pertama
df.shape            # (baris, kolom) → (10, 5)
df.info()           # tipe data & jumlah yang tidak
kosong
df.isna().sum()     # jumlah data kosong per kolom
```

Excel	SQL	pandas
AutoFilter	WHERE	`df[df["Wilayah"] == "Bandung"]`
Sort	ORDER BY	`df.sort_values("Total", ascending=False)`
SUM	SUM()	`df["Total"].sum()`
Pivot / SUMIFS	GROUP BY	`df.groupby("Sales")["Total"].sum()`
COUNTIF tiap nilai	COUNT … GROUP BY	`df["Sales"].value_counts()`
XLOOKUP	LEFT JOIN	`pd.merge(df, produk, on="Produk", how="left")`
Pivot Table	SUM(CASE…)	`pd.pivot_table(df, values="Total", index="Sales", columns="Wilayah", aggfunc="sum")`
Rumus ditarik ke bawah	kolom hitung	`df["Harga"] = df["Total"] / df["Qty"]`

⚠️ Dua syarat filter: `(syarat1) & (syarat2)` — pakai `&` / `|`, bukan `and` / `or`, dan setiap syarat wajib diberi kurung.

🧪 **Praktik:** buka Google Colab, upload `latihan/penjualan.csv` dari proyek ini, lalu ulangi semua contoh di tabel. Kodenya bisa langsung dipakai untuk notebook portofolio skripsimu.

🧠 Uji Diri — 16 soal

DATA YANG DIPAKAI SOAL-SOAL BAB INI

📄 **Sheet: Penjualan**

	A	B	C	D	E	F
1	**Sales**	**Wilayah**	**Produk**	**Qty**	**Harga**	**Total**
2	Andi	Jakarta	Laptop	2	7.000.000	14.000.000
3	Bima	Bandung	Mouse	10	150.000	1.500.000
4	Citra	Jakarta	Monitor	3	2.000.000	6.000.000
5	Andi	Surabaya	Mouse	5	150.000	750.000
6	Dewi	Bandung	Laptop	1	7.000.000	7.000.000
7	Bima	Jakarta	Keyboard	4	300.000	1.200.000
8	Citra	Surabaya	Laptop	1	7.000.000	7.000.000
9	Andi	Jakarta	Monitor	2	2.000.000	4.000.000
10	Dewi	Surabaya	Keyboard	6	300.000	1.800.000
11	Bima	Bandung	Monitor	1	2.000.000	2.000.000

1. **Cara import pandas yang menjadi konvensi di hampir semua tutorial dan perusahaan:**

   A. import pandas as pd
   B. import pandas as pa
   C. from pandas import *
   D. include pandas

2. **pd.read_csv("penjualan.csv") menghasilkan objek bertipe...**

   A. DataFrame — tabel berbaris dan berkolom, mirip sheet Excel
   B. List of strings
   C. Dictionary
   D. File teks biasa

3. 📄 Data: Penjualan

   df berisi data penjualan di atas (10 baris, 5 kolom: Sales, Wilayah, Produk, Qty, Total). Output kode ini:

   ```
   print(df.shape)
   ```

   A. (10, 5)
   B. (5, 10)
   C. 10
   D. 50

4. **df.head() tanpa argumen menampilkan...**

   A. 5 baris pertama
   B. 10 baris pertama
   C. Semua baris
   D. Nama kolom saja

5. 📄 Data: Penjualan

   Output kode ini:

   ```
   print(df["Total"].sum())
   ```

   A. 45250000
   B. 10
   C. 4525000
   D. 14000000

6. 📄 Data: Penjualan

   Output kode ini:

   ```
   bandung = df[df["Wilayah"] == "Bandung"]
   print(len(bandung))
   ```

   A. 3
   B. 4
   C. 10
   D. 1

7. **Cara menulis filter DUA syarat (Wilayah Jakarta DAN Total ≥ 5 juta) yang benar di pandas:**

   A. df[(df["Wilayah"] == "Jakarta") & (df["Total"] >= 5000000)]
   B. df[df["Wilayah"] == "Jakarta" and df["Total"] >= 5000000]
   C. df[df["Wilayah"] == "Jakarta" & df["Total"] >= 5000000]
   D. df.where("Jakarta", 5000000)

8. 📄 Data: Penjualan

   Output kode ini:

```
per_sales = df.groupby("Sales")["Total"].sum()
print(per_sales["Andi"])
```

   A. 18750000
   B. 14000000
   C. 3
   D. 45250000

9. 📄 Data: Penjualan

   Output kode ini:

```
top = df.sort_values("Total", ascending=False)
print(top.iloc[0]["Sales"])
```

   A. Andi
   B. Bima
   C. Dewi
   D. Citra

10. 📄 Data: Penjualan

    Output kode ini:

    ```
    print(df["Sales"].value_counts()["Bima"])
    ```

    A. 3
    B. 4700000
    C. 1
    D. 10

11. 📄 Data: Penjualan

    Output kode ini:

    ```
    df["Harga"] = df["Total"] / df["Qty"]
    print(df["Harga"].iloc[0])
    ```

    A. 7000000.0
    B. 14000000
    C. 2
    D. 7000000

12. **Kamu punya df penjualan (kolom Produk) dan df_produk (kolom Produk, Kategori). Cara menambahkan Kategori ke df penjualan:**

    A. pd.merge(df, df_produk, on="Produk", how="left")
    B. df + df_produk
    C. df.append(df_produk)
    D. df.groupby(df_produk)

13. **Kode untuk menghitung jumlah data KOSONG (missing) di setiap kolom:**

    A. df.isna().sum()
    B. df.count()
    C. df.empty
    D. df.dropna()

14. **Membuat tabel ringkasan seperti Pivot Table Excel (baris: Sales, kolom: Wilayah, nilai: jumlah Total):**

    A. pd.pivot_table(df, values="Total", index="Sales", columns="Wilayah", aggfunc="sum")
    B. df.pivot("Sales")
    C. df.sum("Sales", "Wilayah")
    D. df.sort_values(["Sales", "Wilayah"])

15. 📄 Data: Penjualan

    Output kode ini:

```
r = df.groupby("Wilayah")["Total"].agg(["sum", "count"])
print(r.loc["Jakarta", "count"])
```

    A. 4
    B. 25200000
    C. 3
    D. 10

16. **Fungsi pandas untuk membaca file CSV: pd._____("data.csv")**

    Jawab: _____

Kunci & Pembahasan · Python Bab 12

← Kembali ke soal · Awal bab

1. **A. import pandas as pd** — Alias 'pd' adalah kesepakatan komunitas. Mengikuti konvensi membuat kodemu mudah dibaca orang lain.
   - **B:** Bisa jalan, tapi bukan konvensi — orang lain akan bingung.
   - **C:** Mengimpor semua nama ke ruang global; sulit dilacak asal fungsinya.
   - **D:** Itu sintaks bahasa lain, bukan Python.

2. **A. DataFrame — tabel berbaris dan berkolom, mirip sheet Excel** — DataFrame adalah inti pandas. Setiap kolom adalah Series. Hampir semua yang kamu lakukan di Excel (filter, sort, SUMIFS, pivot, VLOOKUP) bisa dilakukan di DataFrame dengan beberapa baris kode.
   - **B:** pandas sudah memecah baris dan kolom untukmu.
   - **C:** DataFrame bisa dibuat dari dict, tapi hasil read_csv adalah DataFrame.
   - **D:** pandas membaca dan memproses isinya, bukan sekadar teks.

3. **A. (10, 5)** — shape = (jumlah baris, jumlah kolom). Biasakan cek shape setiap selesai filter atau merge — kalau jumlah baris tiba-tiba melonjak, mungkin ada join yang menduplikasi data.
   - **B:** Urutannya (baris, kolom).
   - **C:** shape mengembalikan tuple dua angka.
   - **D:** Itu jumlah sel (df.size).

4. **A. 5 baris pertama** — head(n) menampilkan n baris pertama (default 5); tail() menampilkan baris terakhir. Langkah pertama analisis: head(), shape, info(), dan describe().
   - **B:** Default-nya 5, kecuali kamu menulis head(10).
   - **C:** Untuk data besar, menampilkan semua baris tidak praktis.
   - **D:** Nama kolom: df.columns.
5. **A. 45250000** — df["Total"] mengambil satu kolom (Series), lalu .sum() menjumlahkannya — setara =SUM(F2:F11) di Excel.
   - **B:** 10 adalah jumlah baris (count).
   - **C:** Itu rata-ratanya (mean).
   - **D:** Itu nilai terbesar (max).
6. **A. 3** — df[kondisi] = filter baris (seperti AutoFilter di Excel atau WHERE di SQL). Bagian dalam menghasilkan deretan True/False untuk tiap baris; yang True dipertahankan. Bandung: Bima (Mouse), Dewi (Laptop), Bima (Monitor).
   - **B:** Jakarta yang punya 4 baris.
   - **C:** 10 adalah semua baris sebelum filter.
   - **D:** Filter mempertahankan SEMUA baris yang cocok.

7. **A. df[(df["Wilayah"] == "Jakarta") & (df["Total"] >= 5000000)]** — Di pandas, DAN = &, ATAU = |, TIDAK = ~, dan setiap syarat WAJIB diberi kurung. Hasilnya 2 baris: Andi (14 jt) dan Citra (6 jt).
   - **B:** 'and' Python tidak bisa dipakai untuk Series — muncul error 'truth value of a Series is ambiguous'.
   - **C:** Tanpa kurung, & dikerjakan lebih dulu daripada == sehingga error/salah.
   - **D:** where bekerja dengan cara lain dan tidak menerima argumen seperti itu.
8. **A. 18750000** — groupby + sum = pivot table / SUMIFS per kategori / GROUP BY di SQL. Andi: 14.000.000 + 750.000 + 4.000.000 = 18.750.000.
   - **B:** Itu hanya transaksi pertama Andi.
   - **C:** 3 adalah jumlah transaksi Andi (count).
   - **D:** Itu total semua sales.
9. **A. Andi** — sort_values mengurutkan; ascending=False = dari terbesar. iloc[0] mengambil baris pertama berdasarkan POSISI. Transaksi terbesar: Andi, 14 juta.
   - **B:** Bima tidak punya transaksi terbesar.
   - **C:** Dewi punya 7 juta, bukan yang terbesar.
   - **D:** Citra punya 7 juta, bukan yang terbesar.

10. **A. 3** — value_counts() menghitung kemunculan tiap nilai — seperti COUNTIF untuk setiap nama sekaligus. Bima muncul 3 kali.
    - **B:** Itu total rupiah Bima, bukan banyaknya transaksi.
    - **C:** Bima muncul lebih dari sekali.
    - **D:** 10 adalah jumlah semua baris.

11. **A. 7000000.0** — Operasi antar kolom dikerjakan per baris sekaligus (vectorized) — tanpa loop, seperti rumus yang ditarik ke bawah di Excel. Pembagian / selalu menghasilkan float, jadi tampil 7000000.0.
    - **B:** Itu Total-nya, belum dibagi Qty.
    - **C:** Itu Qty-nya.
    - **D:** Pembagian di Python menghasilkan float, jadi ada .0.

12. **A. pd.merge(df, df_produk, on="Produk", how="left")** — merge = JOIN di SQL = XLOOKUP di Excel. how="left" mempertahankan semua baris penjualan. Cek shape setelahnya: tetap 10 baris, kolom bertambah satu menjadi 6.
    - **B:** Menjumlah dua DataFrame tidak menggabungkan berdasarkan kunci.
    - **C:** Menumpuk ke bawah (dan append sudah dihapus di pandas modern; gunakan pd.concat).
    - **D:** groupby untuk meringkas, bukan menggabungkan.

13. **A. df.isna().sum()** — isna() memberi True untuk sel kosong; .sum() menghitung True per kolom. Ini salah satu langkah wajib di awal analisis. Lalu putuskan: isi (fillna), hapus (dropna), atau selidiki penyebabnya.
    - **B:** count() menghitung yang TIDAK kosong.
    - **C:** empty hanya memberi tahu apakah DataFrame tidak punya baris sama sekali.
    - **D:** dropna menghapus baris yang kosong, bukan menghitungnya.

14. **A. pd.pivot_table(df, values="Total", index="Sales", columns="Wilayah", aggfunc="sum")** — index = Rows, columns = Columns, values = Values, aggfunc = cara meringkas. Contoh: sel Andi–Jakarta = 18.000.000 (sama dengan hasil SUMIFS di bab Excel).
    - **B:** pivot() tidak menerima argumen seperti itu dan tidak bisa meringkas nilai yang dobel.
    - **C:** sum tidak mengelompokkan seperti itu.
    - **D:** sort_values hanya mengurutkan.

15. **A. 4** — agg menjalankan beberapa fungsi ringkasan sekaligus. Jakarta: sum = 25.200.000, count = 4 transaksi. loc mengambil berdasarkan LABEL (baris 'Jakarta', kolom 'count').
    - **B:** Itu nilai kolom 'sum'.
    - **C:** Bandung dan Surabaya yang punya 3 transaksi.
    - **D:** 10 adalah semua transaksi.

16. **read_csv** — pd.read_csv() untuk CSV, pd.read_excel() untuk .xlsx, dan pd.read_sql() untuk hasil query database. Kebalikannya: df.to_csv() dan df.to_excel().

Python untuk Odoo — Model, Field & ORM

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Odoo dibangun dengan Python. Memahami dasarnya membuatmu kuat di dua jalur: **Technical** (menulis modul) dan **Functional** (mengerti apa yang mungkin dan berapa sulitnya).

```
from odoo import models, fields, api

class Kursus(models.Model):
    _name = 'akademi.kursus'          # → tabel
akademi_kursus
    _description = 'Kursus'

    nama      = fields.Char('Nama Kursus',
required=True)
    pengajar  = fields.Many2one('res.partner')
# banyak → satu
    qty       = fields.Integer()
    harga     = fields.Float()
    total     =
fields.Float(compute='_compute_total')

    @api.depends('qty', 'harga')
    def _compute_total(self):
        for rec in self:                  # self =
recordset (bisa banyak)
            rec.total = rec.qty * rec.harga
```

ORM = SQL versi Python:

```
env = self.env['res.partner']
env.search([('city', '=', 'Bandung')])          #
SELECT … WHERE
env.search_count([('customer_rank', '>', 0)])   #
COUNT
env.create({'name': 'Rina'})                    #
INSERT
rec.write({'city': 'Jakarta'})                  #
UPDATE
rec.unlink()                                    #
DELETE
orders.mapped('amount_total')                   #
ambil satu field dari semua record
```

Domain memakai notasi prefix: `['|', ('city','=','Bandung'), ('city','=','Jakarta')]`.

Struktur modul:

```
akademi/
├── __manifest__.py        # nama, versi, depends,
data
├── __init__.py
├── models/kursus.py       # model & logika (Python)
├── views/kursus_views.xml # form, list, menu (XML)
└── security/ir.model.access.csv   # hak akses —
wajib untuk model baru
```

> 🔧 Kustomisasi yang benar memakai **inheritance** (`_inherit = 'sale.order'`), bukan mengedit kode inti. Model penting: `res.partner` , `sale.order` , `purchase.order` , `stock.picking` , `account.move` , `product.template` .

🧠 Uji Diri — 16 soal

1. **Pada model Odoo ini, baris _name = 'akademi.kursus' menentukan...**

```
from odoo import models, fields

class Kursus(models.Model):
    _name = 'akademi.kursus'
    _description = 'Kursus'

    nama  = fields.Char(string='Nama Kursus', required=True)
    harga = fields.Float()
    aktif = fields.Boolean(default=True)
```

   A. Nama teknis model, yang juga menentukan nama tabel di PostgreSQL
   B. Judul halaman di browser
   C. Nama file Python
   D. Nama database

2. **Model dengan _name = 'akademi.kursus' disimpan di tabel PostgreSQL bernama...**

   A. akademi_kursus
   B. akademi.kursus
   C. Kursus
   D. kursus_akademi

3. **Field Odoo yang tepat untuk menghubungkan kursus ke SATU pengajar dari daftar kontak (res.partner):**

   A. fields.Many2one('res.partner')
   B. fields.Char()
   C. fields.One2many('res.partner')
   D. fields.Integer()

4. **Di Odoo, Sales Order dan baris-baris produknya (order lines) berelasi...**

   A. Sales Order → order_line adalah One2many; setiap line → order_id adalah Many2one
   B. Many2many
   C. Tidak berelasi
   D. One2one

5. **Pada computed field ini, kenapa ada perulangan for rec in self?**

```
total = fields.Float(compute='_compute_total')

@api.depends('qty', 'harga')
def _compute_total(self):
    for rec in self:
        rec.total = rec.qty * rec.harga
```

   A. Karena self adalah RECORDSET — bisa berisi banyak record sekaligus (misal di tampilan list)
   B. Karena Python mewajibkan for di setiap fungsi
   C. Supaya lebih lambat
   D. Tidak perlu, hanya gaya penulisan

6. **Fungsi decorator @api.depends('qty', 'harga') pada computed field:**

   A. Memberi tahu Odoo untuk menghitung ulang field saat qty atau harga berubah
   B. Membuat field wajib diisi
   C. Menghapus field
   D. Menyembunyikan field dari user

7. **Kode ini menghasilkan...**

```
pelanggan = self.env['res.partner'].search([('city', '=', 'Bandung')])
```

   A. Recordset berisi semua kontak dengan kota Bandung
   B. Jumlah kontak di Bandung
   C. Satu kontak pertama saja
   D. Teks 'Bandung'

8. **Domain Odoo untuk kontak di Bandung ATAU Jakarta:**

   A. ['|', ('city', '=', 'Bandung'), ('city', '=', 'Jakarta')]
   B. [('city', '=', 'Bandung'), ('city', '=', 'Jakarta')]
   C. [('city', '=', 'Bandung' or 'Jakarta')]
   D. ['or', 'Bandung', 'Jakarta']

9. **Padanan method ORM Odoo dengan SQL:**

   A. create = INSERT, write = UPDATE, unlink = DELETE, search/browse = SELECT
   B. create = SELECT, write = DELETE
   C. unlink = INSERT
   D. Odoo tidak berhubungan dengan SQL

10. **Klien ingin menambah field 'Nomor PO Pelanggan' di Sales Order. Cara standar di modul kustom:**

```
class SaleOrder(models.Model):
    _inherit = 'sale.order'

    no_po_pelanggan = fields.Char('Nomor PO Pelanggan')
```

    A. Memakai _inherit = 'sale.order' untuk MENAMBAH field ke model yang sudah ada
    B. Menyalin seluruh kode modul Sales lalu mengeditnya
    C. Mengubah langsung file inti Odoo
    D. Membuat database baru

11. **Tampilan Odoo (form, list, kanban, pivot, graph, search) didefinisikan dengan...**

    A. XML
    B. Python
    C. CSS saja
    D. Excel

12. **File __manifest__.py sebuah modul Odoo berisi...**

    A. Informasi modul: nama, versi, depends (modul yang dibutuhkan), dan daftar file data (XML/CSV) yang dimuat
    B. Semua data transaksi
    C. Password database
    D. Hanya komentar

13. **Model baru sudah dibuat, tapi user biasa tidak bisa membuka menunya dan log menampilkan peringatan 'no access rules'. Yang terlupa:**

    A. File security/ir.model.access.csv yang memberi hak baca/tulis model itu ke grup user
    B. Restart komputer
    C. Menambah CSS
    D. Mengganti nama class

14. **Invoice pelanggan di Odoo versi modern (13 ke atas) disimpan di model...**

    A. account.move
    B. account.invoice
    C. sale.invoice
    D. res.invoice

15. **Hasil ekspresi orders.mapped('amount_total') adalah...**

```
orders =
self.env['sale.order'].search([('state', '=',
'sale')])
total = sum(orders.mapped('amount_total'))
```

A. List nilai amount_total dari setiap order, sehingga sum() menjumlahkannya
B. Satu angka total
C. Recordset partner
D. Error karena orders berisi banyak record

16. **Lengkapi decorator agar computed field dihitung ulang saat field lain berubah: @api._____('qty', 'harga')**

Jawab: _____

Kunci & Pembahasan · Python Bab 13

← Kembali ke soal · Awal bab

1. **A. Nama teknis model, yang juga menentukan nama tabel di PostgreSQL** — Odoo memakai ORM: setiap class model menjadi tabel. _name = 'akademi.kursus' → tabel akademi_kursus, dan setiap fields.* menjadi kolom. Kamu menulis Python, Odoo yang membuat tabel SQL-nya.
   - **B:** Judul tampilan diatur di view/menu.
   - **C:** Nama file bebas.
   - **D:** Nama database diatur saat instalasi.

2. **A. akademi_kursus** — Titik pada _name diganti garis bawah. Contoh lain: sale.order → sale_order, res.partner → res_partner. Berguna saat kamu perlu menulis query SQL langsung untuk analisis data Odoo.
   - **B:** Nama tabel SQL tidak memakai titik seperti itu.
   - **C:** Nama class Python tidak dipakai sebagai nama tabel.
   - **D:** Urutannya tidak dibalik.

3. **A. fields.Many2one('res.partner')** — Many2one = banyak kursus → satu pengajar (seperti foreign key/pelanggan_id di tabel pesanan). One2many = kebalikannya (satu order → banyak baris order line). Many2many = banyak ke banyak (misal tag).
    - **B:** Char hanya menyimpan teks; tidak terhubung ke data kontak.
    - **C:** One2many untuk satu-ke-banyak, butuh Many2one pasangannya di model lain.
    - **D:** Menyimpan angka id manual kehilangan fitur relasi ORM.
4. **A. Sales Order → order_line adalah One2many; setiap line → order_id adalah Many2one** — Pola header–detail ini ada di mana-mana: invoice & invoice lines, PO & PO lines. Di SQL: tabel sale_order_line punya kolom order_id yang menunjuk ke sale_order.id.
    - **B:** Satu baris produk hanya milik satu order, jadi bukan Many2many.
    - **C:** Mereka terhubung lewat order_id.
    - **D:** Odoo tidak memakai tipe One2one untuk ini.

5. **A. Karena self adalah RECORDSET — bisa berisi banyak record sekaligus (misal di tampilan list)** — Di Odoo, method dipanggil pada kumpulan record. Tanpa loop, kode akan error saat self berisi lebih dari satu record ('Expected singleton'). Ini kesalahan paling umum developer Odoo pemula.
    - **B:** Python tidak mewajibkan for.
    - **C:** Tujuannya kebenaran, bukan kecepatan.
    - **D:** Wajib — tanpa loop akan error 'Expected singleton'.
6. **A. Memberi tahu Odoo untuk menghitung ulang field saat qty atau harga berubah** — Tanpa depends yang benar, nilai computed field bisa 'basi' (tidak ter-update). Mirip rumus Excel yang otomatis berubah saat sel yang dirujuknya diubah.
    - **B:** Wajib diisi diatur dengan required=True.
    - **C:** Menghapus field dilakukan dengan menghapus definisinya.
    - **D:** Menyembunyikan field diatur di view (invisible).

7. **A. Recordset berisi semua kontak dengan kota Bandung** — search(domain) = SELECT ... WHERE. Domain ditulis sebagai list berisi tuple (field, operator, nilai). Untuk jumlahnya saja: search_count(domain). Untuk membatasi: search(domain, limit=1).
    - **B:** Jumlah didapat dari search_count().
    - **C:** search mengembalikan semua yang cocok, kecuali diberi limit.
    - **D:** Yang dikembalikan adalah record kontak.
8. **A. ['|', ('city', '=', 'Bandung'), ('city', '=', 'Jakarta')]** — Domain Odoo memakai notasi prefix (Polish notation): operator '|' (OR) atau '&' (AND) ditulis SEBELUM dua syarat. Tanpa operator, syarat-syarat digabung dengan AND. Alternatif lain: [('city', 'in', ['Bandung', 'Jakarta'])].
    - **B:** Tanpa '|', keduanya digabung AND — tidak ada kota yang sekaligus Bandung dan Jakarta, jadi hasilnya kosong.
    - **C:** 'Bandung' or 'Jakarta' dievaluasi Python menjadi 'Bandung' saja.
    - **D:** Bukan format domain yang valid.

9. **A. create = INSERT, write = UPDATE, unlink = DELETE, search/browse = SELECT** — ORM menerjemahkan method Python menjadi SQL sekaligus menjalankan aturan bisnis (hak akses, computed field, constraint). Karena itu developer Odoo sebaiknya memakai ORM, bukan SQL langsung, untuk menulis data.
    - **B:** Salah padanan.
    - **C:** unlink artinya menghapus.
    - **D:** ORM menghasilkan SQL untuk PostgreSQL.
10. **A. Memakai _inherit = 'sale.order' untuk MENAMBAH field ke model yang sudah ada** — Inheritance adalah inti kustomisasi Odoo: kamu memperluas model yang ada tanpa menyentuh kode intinya, sehingga upgrade versi tetap memungkinkan. Field baru lalu ditampilkan dengan inherit view di XML.
    - **B:** Menyalin kode membuatnya sulit dirawat dan di-upgrade.
    - **C:** Mengubah kode inti akan hilang saat update dan merusak sistem.
    - **D:** Tidak perlu database baru.

11. **A. XML** — Model & logika ditulis di Python, tampilan di XML, interaktivitas frontend di JavaScript (OWL). Catatan: tag tampilan daftar bernama <tree> sampai Odoo 17, dan menjadi <list> mulai Odoo 18.
    - **B:** Python untuk model dan logika bisnis.
    - **C:** CSS hanya untuk gaya tampilan.
    - **D:** Excel bisa dipakai untuk impor data, bukan definisi tampilan.
12. **A. Informasi modul: nama, versi, depends (modul yang dibutuhkan), dan daftar file data (XML/CSV) yang dimuat** — Contoh: 'depends': ['sale'] berarti modulmu butuh Sales ter-install. Lupa mendaftarkan file XML di 'data' adalah penyebab umum 'view saya tidak muncul'.
    - **B:** Data transaksi ada di database.
    - **C:** Password tidak boleh disimpan di kode modul.
    - **D:** Manifest adalah dictionary Python yang dibaca Odoo.
13. **A. File security/ir.model.access.csv yang memberi hak baca/tulis model itu ke grup user** — Setiap model baru butuh aturan akses (read, write, create, unlink) per grup, dan file CSV-nya harus didaftarkan di manifest. Tanpa itu, hanya superuser yang bisa mengaksesnya.
    - **B:** Restart tidak menambah hak akses.
    - **C:** CSS tidak berhubungan dengan hak akses.
    - **D:** Nama class tidak memengaruhi hak akses.

14. **A. account.move** — Sejak Odoo 13, invoice, vendor bill, dan jurnal umum digabung di account.move (barisnya: account.move.line). Model penting lain: res.partner (kontak), sale.order, purchase.order, stock.picking (pengiriman/penerimaan), product.template/product.product.
    - **B:** account.invoice adalah model lama (Odoo 12 ke bawah).
    - **C:** Tidak ada model ini.
    - **D:** Tidak ada model ini.
15. **A. List nilai amount_total dari setiap order, sehingga sum() menjumlahkannya** — mapped mengambil satu field dari semua record dalam recordset. Pola ini sering dipakai untuk KPI di dashboard: total penjualan yang sudah dikonfirmasi (state = 'sale').
    - **B:** Angka totalnya baru didapat setelah sum().
    - **C:** Itu terjadi jika mapped berisi field relasi seperti 'partner_id'.
    - **D:** mapped justru dirancang untuk recordset berisi banyak record.
16. **depends** — @api.depends. Decorator lain yang sering dipakai: @api.onchange (reaksi di form sebelum disimpan) dan @api.constrains (validasi saat disimpan).

Latihan Membaca Kode (Gaya Tes Coding)

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Tes coding untuk fresh graduate sering berbentuk "apa output kode ini?" atau soal logika kecil. Kemampuan ini dilatih dengan **menelusuri kode baris demi baris**, bukan menghafal.

Teknik trace table — tulis nilai variabel setiap langkah:

```
total = 0
for i in range(1, 6):
    if i % 2 == 0: continue
    total += i

 i | genap? | total
 1 | tidak  | 1
 2 | ya     | 1   (dilewati)
 3 | tidak  | 4
 4 | ya     | 4   (dilewati)
 5 | tidak  | 9   → output 9
```

Pola yang paling sering keluar di tes:

- Menghitung frekuensi dengan dict: `d[k] = d.get(k, 0) + 1`
- Mengurutkan dengan `key=lambda x: …`
- Filter dengan list comprehension: `[x for x in data if x >= 6]`
- Nilai unik: `len(set(data))`
- `//` dan `%`, palindrom dengan `s[::-1]`, FizzBuzz

- Jebakan: `b = a` bukan salinan; fungsi tanpa `return` menghasilkan `None`

> 💡 Setelah menjawab, jalankan kodenya sendiri di Google Colab atau `python` di terminal. Membandingkan tebakanmu dengan hasil sebenarnya adalah cara tercepat melatih "insting" membaca kode.

🧠 Uji Diri — 16 soal

1. **Output kode ini:**

```
angka = [3, 8, 2, 8, 5]
print(max(angka), angka.index(8))
```

A. 8 1
B. 8 3
C. 8 [1, 3]
D. 5 1

2. **Output kode ini:**

```
total = 0
for i in range(1, 6):
    if i % 2 == 0:
        continue
    total += i
print(total)
```

A. 9
B. 15
C. 6
D. 5

3. **Output kode ini:**

```
kata = "data analis data bisnis data"
hitung = {}
for k in kata.split():
    hitung[k] = hitung.get(k, 0) + 1
print(hitung["data"])
```

A. 3
B. 1
C. 5
D. Error KeyError

4. **Output kode ini:**

```
sales = [("Andi", 18.75), ("Bima", 4.7),
("Citra", 13.0)]
terbaik = sorted(sales, key=lambda x: x[1],
reverse=True)
print(terbaik[0][0])
```

A. Andi
B. Citra
C. Bima
D. 18.75

5. **Output kode ini:**

```
total = [14, 1.5, 6, 0.75, 7]
besar = [t for t in total if t >= 6]
print(len(besar))
```

A. 3
B. 2
C. 5
D. 4

6. **Output kode ini:**

```
a = [1, 2, 3]
b = a
b.append(4)
print(a)
```

A. [1, 2, 3, 4]
B. [1, 2, 3]
C. [4]
D. Error

7. **Output kode ini:**

```
s = "katak"
print(s == s[::-1])
```

A. True
B. False
C. katak
D. kataK

8. **Output kode ini:**

```
for i in range(1, 16):
    if i % 15 == 0:
        hasil = "FizzBuzz"
    elif i % 3 == 0:
        hasil = "Fizz"
    elif i % 5 == 0:
        hasil = "Buzz"
    else:
        hasil = str(i)
print(hasil)
```

A. FizzBuzz

B. Fizz

C. Buzz

D. 15

9. **Output kode ini:**

```
nama = ["Andi", "Bima"]
target = [20, 10]
d = dict(zip(nama, target))
print(d["Bima"])
```

A. 10

B. 20

C. Bima

D. Error

10. **Output kode ini:**

```
n = 0
for i in range(3):
    for j in range(i):
        n += 1
print(n)
```

A. 3
B. 9
C. 6
D. 0

11. **Output kode ini:**

```
menit = 135
print(menit // 60, menit % 60)
```

A. 2 15
B. 2.25 15
C. 2 0.25
D. 135 60

12. **Output kode ini:**

```
kota = ["Jakarta", "Bandung", "Jakarta",
"Surabaya", "Bandung"]
print(len(set(kota)))
```

A. 3
B. 5
C. 2
D. 1

13. **Saldo tabungan 100 bertambah 10% per bulan. Output kode ini:**

```
saldo = 100
bulan = 0
while saldo < 150:
    saldo = saldo * 1.1
    bulan += 1
print(bulan)
```

A. 5
B. 4
C. 6
D. 50

14. **Output kode ini:**

```
def tambah(a, b):
    print(a + b)

hasil = tambah(2, 3)
print(hasil)
```

A. 5 None
B. 5 5
C. None
D. 5

15. **Output kode ini:**

```
try:
    x = int("12a")
except ValueError:
    x = 0
print(x)
```

A. 0
B. 12
C. Error ValueError
D. 12a

16. **Output kode ini:**

```
for no, nama in enumerate(["Andi", "Bima",
"Citra"], start=1):
    pass
print(no, nama)
```

A. 3 Citra
B. 2 Citra
C. 1 Andi
D. Error

Kunci & Pembahasan · Python Bab 14

← Kembali ke soal · Awal bab

1. **A. 8 1** — max → 8. index() mengembalikan posisi kemunculan PERTAMA, dan indeks dimulai dari 0: angka[1] = 8.
   - **B:** index mengembalikan kemunculan pertama, bukan terakhir.
   - **C:** index hanya mengembalikan satu angka.
   - **D:** Nilai terbesar adalah 8.
2. **A. 9** — range(1, 6) = 1..5. continue melompati angka genap (2 dan 4). Yang dijumlah: 1 + 3 + 5 = 9.
   - **B:** 15 = semua angka 1..5; padahal yang genap dilewati.
   - **C:** 6 = jumlah angka genapnya (2 + 4).
   - **D:** 5 hanya angka terakhir.
3. **A. 3** — Pola menghitung frekuensi yang sangat sering muncul di tes coding. get(k, 0) mengembalikan 0 jika kunci belum ada, jadi tidak terjadi KeyError. Kata 'data' muncul 3 kali. (Versi singkatnya: collections.Counter.)
   - **B:** Setiap kemunculan menambah 1.
   - **C:** 5 adalah jumlah semua kata.
   - **D:** get(k, 0) mencegah KeyError.

4. **A. Andi** — key=lambda x: x[1] berarti 'urutkan berdasarkan elemen ke-1 (angka)'. reverse=True = dari terbesar. terbaik[0] = ("Andi", 18.75), lalu [0] mengambil namanya.
   - **B:** Citra di urutan kedua.
   - **C:** Bima paling kecil.
   - **D:** [0][0] mengambil nama, bukan angka.

5. **A. 3** — List comprehension dengan filter: ambil t yang ≥ 6 → [14, 6, 7]. Panjangnya 3. Setara FILTER di Excel atau WHERE di SQL.
   - **B:** 6 juga memenuhi syarat ≥ 6.
   - **C:** Tidak semua nilai ≥ 6.
   - **D:** 1,5 dan 0,75 tidak lolos.

6. **A. [1, 2, 3, 4]** — b = a TIDAK menyalin list; a dan b menunjuk objek yang SAMA. Mengubah b ikut mengubah a. Untuk menyalin: b = a.copy() atau b = a[:]. Sumber bug yang sangat umum.
   - **B:** Itu jika b = a.copy().
   - **C:** append menambah ke list yang sudah ada.
   - **D:** Kode ini valid.

7. **A. True** — s[::-1] membalik string (slicing dengan langkah −1). 'katak' dibalik tetap 'katak' → palindrom → True. Soal palindrom sangat sering muncul di tes coding pemula.
   - **B:** Dibalik pun tetap sama.
   - **C:** Yang dicetak hasil perbandingan (bool), bukan string.
   - **D:** Huruf tidak berubah saat dibalik.

8. **A. FizzBuzz** — print berada DI LUAR loop (tidak menjorok), jadi hanya mencetak nilai terakhir, yaitu saat i = 15 → 'FizzBuzz'. Perhatikan juga urutan syaratnya: % 15 harus dicek lebih dulu — sama seperti IF bertingkat dan CASE WHEN.
    - **B:** 15 juga habis dibagi 3, tapi syarat % 15 dicek lebih dulu.
    - **C:** 15 juga habis dibagi 5, tapi syarat % 15 dicek lebih dulu.
    - **D:** 15 hanya dicetak jika tidak habis dibagi 3 maupun 5.
9. **A. 10** — zip memasangkan elemen dengan posisi yang sama: ("Andi", 20), ("Bima", 10). dict() mengubahnya menjadi kamus — seperti tabel lookup kecil.
    - **B:** 20 adalah target Andi.
    - **C:** Yang diambil adalah nilai, bukan kuncinya.
    - **D:** Kunci 'Bima' ada.
10. **A. 3** — Loop dalam berjalan i kali: saat i = 0 → 0 kali, i = 1 → 1 kali, i = 2 → 2 kali. Total 0 + 1 + 2 = 3. Latih membaca loop bersarang langkah demi langkah dengan tabel kecil di kertas.
    - **B:** 9 jika loop dalam selalu 3 kali (range(3)).
    - **C:** 6 jika i berjalan 1..3.
    - **D:** Loop dalam tetap berjalan saat i = 1 dan i = 2.

11. **A. 2 15** — // = pembagian bulat (2 jam), % = sisa bagi (15 menit). Pasangan ini dipakai untuk konversi waktu, membagi halaman, dan mengecek genap/ganjil.
    - **B:** // membuang bagian desimal.
    - **C:** % menghasilkan sisa dalam menit, bukan pecahan jam.
    - **D:** Itu hanya mencetak angkanya.
12. **A. 3** — set membuang duplikat → {Jakarta, Bandung, Surabaya}. Jumlah nilai unik = 3. Setara COUNTA(UNIQUE()) dan COUNT(DISTINCT).
    - **B:** 5 adalah jumlah semua elemen termasuk duplikat.
    - **C:** Ada 3 kota berbeda.
    - **D:** Ada lebih dari satu kota.
13. **A. 5** — 100 → 110 → 121 → 133,1 → 146,41 → 161,05. Setelah 4 bulan masih 146,41 (< 150), jadi butuh 5 bulan. Gunakan while saat jumlah perulangan belum diketahui.
    - **B:** Setelah 4 bulan saldonya 146,41 — belum 150.
    - **C:** Saldo sudah melewati 150 di bulan ke-5.
    - **D:** Bunga majemuk tidak bertambah 1 per bulan.

14. **A. 5 None** — Fungsi ini MENCETAK tapi tidak MENGEMBALIKAN nilai (tidak ada return), sehingga hasilnya None. Bedakan print (menampilkan ke layar) dan return (memberikan nilai ke pemanggil).
    - **B:** Tanpa return, hasil bernilai None.
    - **C:** print(a + b) di dalam fungsi tetap mencetak 5 lebih dulu.
    - **D:** print(hasil) juga mencetak sesuatu: None.
15. **A. 0** — int("12a") gagal karena ada huruf → ValueError → masuk except → x = 0. Pola ini dipakai saat membersihkan data angka yang kotor.
    - **B:** int tidak mengambil angka di depan secara otomatis.
    - **C:** Error-nya ditangkap oleh except.
    - **D:** x bernilai angka 0 setelah except.
16. **A. 3 Citra** — enumerate memberi nomor sekaligus isinya; start=1 membuat nomor dimulai dari 1. Variabel loop tetap ada setelah loop selesai dan berisi nilai terakhir: (3, 'Citra').
    - **B:** Itu jika start=0 (default).
    - **C:** Itu nilai pada putaran pertama.
    - **D:** Variabel loop masih bisa diakses setelah loop di Python.

BAGIAN 2

JavaScript

★ Prioritas #3 — Dashboard web, frontend Odoo (OWL), dan proyek SalesKu.

Dasar, DOM, async, ES6+ modern, jebakan interview

10 bab · 130 soal

Dasar: Variabel, Tipe & Console

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

JavaScript menghidupkan halaman web. Variabel modern:

```
let umur = 22;        // bisa diubah
const NAMA = "Ryan";  // tetap
console.log(umur);    // cek output di DevTools (F12)
```

Tipe utama: string, number, boolean (`true/false`), `undefined` (belum diisi), `null` (sengaja kosong). Cek tipe dengan `typeof`. Konversi: `Number("7")`, `String(7)`. Hindari `var` — scope-nya bocor.

🧠 Uji Diri — 14 soal

1. **Menampilkan pesan ke console browser:**

   A. print()
   B. console.log()
   C. echo()
   D. msg()

2. **Kata kunci modern untuk variabel yang bisa berubah:**

   A. var saja
   B. let
   C. const
   D. int

3. **const digunakan untuk...**

   A. nilai yang tidak di-assign ulang
   B. variabel global
   C. loop
   D. fungsi saja

4. **Komentar satu baris di JavaScript:**

   A. # komentar
   B. <!-- -->
   C. // komentar
   D. ** komentar

5. **Boolean di JavaScript ditulis...**

   A. True/False
   B. true/false
   C. TRUE/FALSE
   D. 1/0 saja

6. **typeof "halo" menghasilkan...**

   A. text
   B. str
   C. string
   D. char

7. **Kotak dialog peringatan di browser:**

   A. alert()
   B. popup()
   C. dialog()
   D. warn()

8. **Mengubah "7" menjadi angka:**

    A. Number("7")
    B. int("7")
    C. toNumber("7")
    D. num("7")

9. **Perbedaan null dan undefined:**

    A. sama
    B. null sengaja dikosongkan; undefined belum diberi nilai
    C. undefined disengaja
    D. null untuk angka

10. **typeof null menghasilkan...**

    A. "null"
    B. "undefined"
    C. "object"
    D. "none"

11. **let umur; console.log(umur) menampilkan...**

    A. 0
    B. null
    C. undefined
    D. error

12. **Kenapa let/const lebih baik dari var?**

    A. lebih pendek
    B. scope per-blok, tak bocor keluar { }
    C. lebih cepat
    D. warna beda

13. **NaN singkatan dari Not a ____**

    Jawab: _____

14. **Deklarasi konstanta: ____ PI = 3.14;**

    Jawab: _____

Kunci & Pembahasan · JavaScript Bab 1

← Kembali ke soal · Awal bab

1. **B. console.log()** — console.log() menulis ke console DevTools.
2. **B. let** — let bisa diubah; const tetap.
3. **A. nilai yang tidak di-assign ulang** — const tidak bisa di-assign ulang.
4. **C. // komentar** — // satu baris; /* */ banyak baris.
5. **B. true/false** — Huruf kecil semua: true, false.
6. **C. string** — Teks bertipe "string".
7. **A. alert()** — alert() memunculkan dialog pesan.
8. **A. Number("7")** — Number(), parseInt(), atau parseFloat().
9. **B. null sengaja dikosongkan; undefined belum diberi nilai** — undefined = belum di-assign; null = sengaja kosong.
10. **C. "object"** — Bug historis JS yang dipertahankan.
11. **C. undefined** — Variabel tanpa nilai = undefined.
12. **B. scope per-blok, tak bocor keluar { }** — var ber-scope fungsi & bisa bocor; let/const block-scoped.
13. **number** — NaN muncul saat operasi angka gagal.
14. **const** — const untuk nilai tetap.

Operator & Perbandingan

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Aritmetika standar `+ - * / %`, tapi hati-hati: `5 + "5" = "55"` (string!), sedangkan `"5" - 2 = 3`.

```
5 ==  "5"   // true  (loose, konversi dulu)
5 === "5"   // false (strict: nilai + tipe)
```

Selalu pakai ===. Logika: `&& || !`. Singkat: `x++`, `x += 3`. Ternary: `status = n >= 75 ? "Lulus" : "Gagal"`. Modern: `a ?? "default"` — fallback khusus null/undefined.

🧠 Uji Diri — 12 soal

1. **Hasil 5 + "5":**

   A. 10
   B. "55"
   C. error
   D. NaN

2. **Operator pembanding nilai DAN tipe:**

   A. ==
   B. ===
   C. =
   D. !=

3. **Hasil 5 == "5":**

   A. true
   B. false
   C. error
   D. NaN

4. **Hasil 5 === "5":**

   A. true
   B. false
   C. error
   D. 5

5. **Operator logika "dan":**

   A. and
   B. &&
   C. &
   D. AND

6. **Hasil 10 % 4:**

   A. 2
   B. 2.5
   C. 0
   D. 4

7. **x = 5; x++; nilai x sekarang...**

   A. 5
   B. 6
   C. 4
   D. 55

8. **Hasil !true:**

   A. true
   B. false
   C. 1
   D. error

9. **Hasil 0.1 + 0.2 === 0.3:**

   A. true
   B. false
   C. error
   D. NaN

10. **a ?? "default" mengembalikan "default" jika a bernilai...**

    A. 0
    B. null atau undefined
    C. ""
    D. false

11. **Hasil "5" - 2:**

    A. 3
    B. "52"
    C. NaN
    D. error

12. **Operator ternary: kondisi ____ nilaiBenar : nilaiSalah**

    Jawab: _____

Kunci & Pembahasan · JavaScript Bab 2

← Kembali ke soal · Awal bab

1. **B. "55"** — Angka + string = penggabungan string.
2. **B. ===** — === strict equality — selalu gunakan ini.
3. **A. true** — == mengonversi tipe dulu (loose) → true.
4. **B. false** — Tipe berbeda (number vs string) → false.
5. **B. &&** — && (AND), || (OR), ! (NOT).
6. **A. 2** — % = sisa bagi: 10÷4 sisa 2.
7. **B. 6** — ++ menambah 1.
8. **B. false** — ! membalik boolean.
9. **B. false** — Floating point: 0.30000000000000004 → false.
10. **B. null atau undefined** — ?? (nullish coalescing) hanya untuk null/undefined.
11. **A. 3** — Operator - memaksa konversi ke angka: 3. (Beda dengan +).
12. **?** — kondisi ? a : b — if-else singkat.

Kondisi & Perulangan

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
if (nilai >= 80) { ... }
else if (nilai >= 70) { ... }
else { ... }

for (let i = 0; i < 3; i++) { ... }  // 0 1 2
for (const buah of daftar) { ... }   // tiap NILAI
while (kondisi) { ... }
```

`break` berhenti; `continue` lompat. `switch…case` untuk banyak cabang — jangan lupa `break` tiap case! Falsy: `0, "", null, undefined, NaN, false` (catatan: `"0"` dan `[]` justru truthy).

🧠 Uji Diri — 13 soal

1. **Struktur if yang benar:**

   A. if x > 5 then
   B. if (x > 5) { }
   C. if x > 5:
   D. when (x>5)

2. **Cabang alternatif kedua ditulis…**

   A. elif
   B. else if
   C. elseif
   D. or if

3. **for (let i=0; i<3; i++) berjalan berapa kali?**

   A. 2
   B. 3
   C. 4
   D. tak hingga

4. **Loop yang mengecek kondisi SETELAH eksekusi pertama:**

   A. for
   B. while
   C. do...while
   D. foreach

5. **Menghentikan loop lebih awal:**

   A. stop
   B. break
   C. exit
   D. end

6. **Loop untuk mengiterasi NILAI array:**

   A. for...of
   B. for...in untuk nilai
   C. loop()
   D. each()

7. **Struktur untuk banyak cabang berdasar satu nilai:**

   A. multi-if saja
   B. switch...case
   C. match
   D. select

8. **Nilai falsy di JavaScript:**

   A. "0"
   B. []
   C. 0
   D. "false"

9. **Kenapa break penting di tiap case switch?**

   A. dekorasi
   B. tanpa break eksekusi 'jatuh' ke case berikutnya
   C. wajib sintaks
   D. mempercepat

10. **let s=0; for(let i=1;i<=3;i++){s+=i}. Nilai s:**

    A. 3
    B. 6
    C. 9
    D. 4

11. **for...in pada objek mengiterasi...**

    A. nilai
    B. kunci (property name)
    C. indeks angka
    D. method

12. **Kata kunci melewati satu iterasi loop: ____**

    Jawab: _____

13. **Cabang default pada switch ditulis ____ :**

    Jawab: _____

Kunci & Pembahasan · JavaScript Bab 3

← Kembali ke soal · Awal bab

1. **B. if (x > 5) { }** — Kondisi dalam ( ), blok dalam { }.
2. **B. else if** — JS memakai else if (dua kata).
3. **B. 3** — i = 0,1,2 → 3 kali.
4. **C. do...while** — do...while pasti jalan minimal sekali.
5. **B. break** — break keluar; continue lompat iterasi.
6. **A. for...of** — for...of nilai; for...in indeks/kunci.
7. **B. switch...case** — switch(x){ case 1: ... break; default: }
8. **C. 0** — Falsy: 0, "", null, undefined, NaN, false. "0" dan [] truthy!
9. **B. tanpa break eksekusi 'jatuh' ke case berikutnya** — Fall-through: tanpa break, case selanjutnya ikut jalan.
10. **B. 6** — 1+2+3 = 6.
11. **B. kunci (property name)** — for (let key in obj) memberi nama properti.
12. **continue** — continue lompat ke iterasi berikutnya.
13. **default** — default menangani nilai yang tak cocok case manapun.

Fungsi & Arrow Function

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

```
function luas(p, l = 1) {   // default param
  return p * l;
}
const kali2 = (x) => x * 2; // arrow: auto-return
```

Tanpa return → `undefined`. Variabel dalam fungsi = lokal. Konsep penting: **callback** (fungsi dikirim ke fungsi lain, mis. `map(fn)`) dan **closure** (fungsi mengingat scope tempat ia lahir). `...rest` menampung argumen bebas sebagai array.

🧠 Uji Diri — 12 soal

1. **Membuat fungsi bernama sapa:**

   A. def sapa():
   B. function sapa() { }
   C. func sapa()
   D. fn sapa()

2. **Mengembalikan nilai dari fungsi:**

   A. give
   B. return
   C. send
   D. out

3. **Arrow function setara function tambah(a,b) {return a+b}:**

   A. const tambah=(a,b)⇒a+b
   B. tambah⇒(a,b)
   C. const tambah=a,b→a+b
   D. arrow tambah()

4. **function f(x=10){ } — 10 adalah...**

   A. batas
   B. parameter default
   C. konstanta
   D. global

5. **Fungsi tanpa return mengembalikan...**

   A. 0
   B. null
   C. undefined
   D. ""

6. **const f = function(){ } disebut...**

   A. function declaration
   B. function expression
   C. arrow
   D. class

7. **Fungsi yang dikirim sebagai argumen fungsi lain disebut...**

   A. callback
   B. promise
   C. closure
   D. helper

8. **Closure adalah...**

   A. fungsi yang mengingat scope tempat ia dibuat
   B. penutup tag
   C. event
   D. loop

9. **Rest parameter function f(...angka) menampung...**

   A. satu angka
   B. semua argumen sebagai array
   C. string
   D. objek

10. **Variabel yang dideklarasi dalam fungsi...**

    A. global
    B. hanya hidup di dalam fungsi (lokal)
    C. permanen
    D. milik window

11. **Simbol panah pada arrow function: (a, b) ____ a + b**

    Jawab: _____

12. **Fungsi yang langsung dieksekusi saat didefinisikan disingkat ____E**

    Jawab: _____

Kunci & Pembahasan · JavaScript Bab 4

← Kembali ke soal · Awal bab

1. **B. function sapa() { }** — function sapa(){ } atau arrow function.
2. **B. return** — return menghentikan fungsi & mengirim nilai.
3. **A. const tambah=(a,b)⇒a+b** — Satu ekspresi otomatis ter-return.
4. **B. parameter default** — Dipakai jika argumen tidak diberikan.
5. **C. undefined** — Default return JS = undefined.
6. **B. function expression** — Fungsi disimpan dalam variabel = expression.
7. **A. callback** — Contoh: arr.map(callback).
8. **A. fungsi yang mengingat scope tempat ia dibuat** — Fungsi dalam tetap bisa akses variabel fungsi luar.
9. **B. semua argumen sebagai array** — ...angka mengumpulkan argumen jadi array.
10. **B. hanya hidup di dalam fungsi (lokal)** — Scope lokal — tak bisa diakses dari luar.
11. **⇒** — ⇒ memisahkan parameter dan badan fungsi.
12. **iif** — IIFE: (function(){ })();

Array & Method Andalannya

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Method array adalah senjata sehari-hari programmer JS:

```
const n = [1, 2, 3, 4];
n.push(5); n.pop();
n.map(x => x * 2)        // [2,4,6,8]  transformasi
n.filter(x => x > 2)     // [3,4]      saring
n.find(x => x > 2)       // 3          satu pertama
n.reduce((a,b) => a+b,0) // 10         rangkum
n.includes(3)            // true
```

Awas: `sort()` default membanding string — untuk angka: `sort((a,b) => a-b)`. `slice` menyalin, `splice` mengubah array asli.

🧠 Uji Diri — 15 soal

1. **Membuat array:**

   A. a=(1,2,3)
   B. a={1,2,3}
   C. a=[1,2,3]
   D. a=<1,2,3>

2. **buah[0] mengambil…**

   A. elemen kedua
   B. elemen pertama
   C. panjang
   D. error

3. **Jumlah elemen array:**

   A. size
   B. count
   C. length
   D. len

4. **Menambah elemen di AKHIR array:**

   A. append()
   B. push()
   C. add()
   D. insert()

5. **Menghapus elemen TERAKHIR:**

   A. remove()
   B. delete()
   C. pop()
   D. shift()

6. **[1,2,3].map(x ⇒ x*2) menghasilkan...**

   A. [2,4,6]
   B. [1,2,3,2]
   C. 6
   D. [1,4,9]

7. **filter() menghasilkan...**

   A. satu nilai
   B. array baru berisi elemen yang lolos kondisi
   C. urutan baru
   D. string

8. **Method mencari SATU elemen pertama yang cocok:**

   A. search()
   B. find()
   C. get()
   D. locate()

9. **arr.includes(5) mengembalikan...**

   A. indeks
   B. true/false apakah 5 ada
   C. elemen
   D. jumlah

10. **[1,2,3].reduce((a,b)⇒a+b, 0) menghasilkan...**

    A. [6]
    B. 6
    C. 123
    D. error

11. **Perbedaan map dan forEach:**

    A. sama
    B. map mengembalikan array baru; forEach tidak
    C. forEach lebih baru
    D. map lebih lambat selalu

12. **arr.slice(1,3) vs arr.splice(1,2):**

    A. sama
    B. slice menyalin tanpa mengubah; splice memotong array asli
    C. splice menyalin
    D. slice menghapus

13. **[3,1,2].sort() lalu angka besar bisa salah urut karena...**

    A. bug browser
    B. sort default membanding sebagai STRING
    C. array rusak
    D. perlu loop

14. **Menggabung array jadi string: arr.____(", ")**

    Jawab: _____

15. **Mengecek apakah SEMUA elemen lolos kondisi: arr.____(x ⇒ x>0)**

    Jawab: _____

Kunci & Pembahasan · JavaScript Bab 5

← Kembali ke soal · Awal bab

1. **C. a=[1,2,3]** — Array memakai kurung siku.
2. **B. elemen pertama** — Indeks mulai dari 0.
3. **C. length** — arr.length (properti, bukan fungsi).
4. **B. push()** — push() akhir; unshift() awal.
5. **C. pop()** — pop() akhir; shift() awal.
6. **A. [2,4,6]** — map membuat array baru hasil transformasi.
7. **B. array baru berisi elemen yang lolos kondisi** — [1,2,3,4].filter(x⇒x>2) → [3,4].
8. **B. find()** — find mengembalikan elemen; findIndex indeksnya.
9. **B. true/false apakah 5 ada** — includes mengecek keberadaan nilai.
10. **B. 6** — reduce merangkum array jadi satu nilai.
11. **B. map mengembalikan array baru; forEach tidak** — forEach hanya menjalankan efek, return undefined.
12. **B. slice menyalin tanpa mengubah; splice memotong array asli** — slice = non-destruktif; splice mengubah array asli.
13. **B. sort default membanding sebagai STRING** — Gunakan sort((a,b)⇒a-b) untuk angka.

14. **join** — join menggabungkan dengan pemisah.

15. **every** — every semua; some minimal satu.

String, Objek & Destructuring

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Template literal — cara modern merangkai teks:

```
console.log(`Halo ${nama}, skor ${skor}`);
```

Method string: `toUpperCase()` `trim()` `split(",")` `replace()` `includes()`. Objek — struktur data terpenting JS:

```
const mhs = { nama: "Ryan", nim: "220103091" };
mhs.nama;               // akses
const { nama } = mhs;   // destructuring
Object.keys(mhs);       // ["nama","nim"]
const salinan = { ...mhs, kota: "Solo" };  // spread
```

🧠 Uji Diri — 12 soal

1. **Template literal yang benar:**

   A. "Halo ${n}"
   B. 'Halo ${n}'
   C. `Halo ${n}`
   D. "Halo"+{n}

2. **"halo".length menghasilkan...**

   A. 3
   B. 4
   C. 5
   D. error

3. **"a,b,c".split(",") menghasilkan...**

   A. "abc"
   B. ["a","b","c"]
   C. (a,b,c)
   D. error

4. **"HALO".toLowerCase() menghasilkan...**

   A. HALO
   B. halo
   C. Halo
   D. error

5. **Membuat objek:**

   A. o=[nama:"R"]
   B. o={nama:"Ryan"}
   C. o=(nama="R")
   D. object o

6. **Mengakses properti nama dari objek o:**

   A. o→nama
   B. o.nama atau o["nama"]
   C. o(nama)
   D. get o.nama

7. **" hai ".trim() menghasilkan...**

   A. " hai "
   B. "hai"
   C. "h a i"
   D. error

8. **Object.keys({a:1,b:2}) menghasilkan...**

   A. [1,2]
   B. ["a","b"]
   C. {a,b}
   D. 2

9. **Destructuring const {nama} = orang; artinya...**

   A. hapus nama
   B. ambil properti nama jadi variabel
   C. salin objek
   D. bandingkan

10. **"belajar".includes("ajar") menghasilkan...**

    A. true
    B. false
    C. 2
    D. "ajar"

11. **Mengganti kata: s.____("a", "b")**

    Jawab: _____

12. **Menyalin isi objek/array ke wadah baru memakai operator titik tiga: [____arr]**

    Jawab: _____

Kunci & Pembahasan · JavaScript Bab 6

← Kembali ke soal · Awal bab

1. **C. `Halo ${n}`** — Backtick (`) + ${ } menyisipkan variabel.
2. **B. 4** — 4 karakter.
3. **B. ["a","b","c"]** — split memecah string jadi array.
4. **B. halo** — toLowerCase/toUpperCase mengubah kapitalisasi.
5. **B. o={nama:"Ryan"}** — Objek: pasangan kunci: nilai dalam { }.
6. **B. o.nama atau o["nama"]** — Dot notation atau bracket notation.
7. **B. "hai"** — trim menghapus spasi awal & akhir.
8. **B. ["a","b"]** — keys → array kunci; values → array nilai.
9. **B. ambil properti nama jadi variabel** — Destructuring mengekstrak properti langsung.
10. **A. true** — includes mengecek substring.
11. **replace** — replace mengganti kemunculan pertama; replaceAll semua.
12. **...** — Spread operator (...) menyalin elemen.

DOM: Menghidupkan Halaman

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

DOM = representasi HTML yang bisa dimanipulasi JS:

```
const btn = document.querySelector("#tombol");
btn.addEventListener("click", () => {
  const el = document.querySelector(".hasil");
  el.textContent = "Diklik!";
  el.classList.toggle("aktif");
});
const isi = document.querySelector("input").value;
```

`textContent` = teks polos (aman); `innerHTML` merender tag (hati-hati input pengguna). Event penting: `click, input, submit, keydown`. `e.preventDefault()` mencegah form me-reload halaman — inilah dasar aplikasi interaktif.

🧠 Uji Diri — 12 soal

1. **Mengambil elemen ber-id "judul":**
   A. document.getElementById("judul")
   B. find("#judul")
   C. get("judul")
   D. html.id("judul")

2. **document.querySelector(".kartu") memilih...**
   A. semua .kartu
   B. elemen PERTAMA ber-class kartu
   C. id kartu
   D. tag kartu

3. **Memasang event klik pada tombol btn:**

   A. btn.onClick()
   B. btn.addEventListener("click", fn)
   C. btn.click=fn()
   D. listen(btn)

4. **Mengubah teks elemen el:**

   A. el.text="Hai"
   B. el.textContent="Hai"
   C. el.write("Hai")
   D. el=Hai

5. **Perbedaan textContent dan innerHTML:**

   A. sama
   B. innerHTML merender tag HTML; textContent teks polos
   C. textContent lebih lambat
   D. innerHTML teks saja

6. **Menambah class "aktif" pada elemen:**

   A. el.class="aktif"
   B. el.classList.add("aktif")
   C. el.addClass()
   D. class(el)

7. **Mengambil nilai ketikan dari input inp:**

   A. inp.text
   B. inp.value
   C. inp.content
   D. inp.data

8. **Membuat elemen baru:**

   A. document.createElement("div")
   B. new Element()
   C. document.add("div")
   D. el.make()

9. **event.preventDefault() berguna untuk...**

   A. menghapus event
   B. mencegah aksi bawaan (mis. form reload)
   C. mempercepat
   D. menghentikan JS

10. **Event yang cocok mendeteksi ketikan real-time di input:**

    A. click
    B. input
    C. load
    D. scroll

11. **Objek global yang mewakili halaman HTML: ____**

    Jawab: _____

12. **classList.____("gelap") menambah/menghapus class bergantian**

    Jawab: _____

Kunci & Pembahasan · JavaScript Bab 7

← Kembali ke soal · Awal bab

1. **A. document.getElementById("judul")** — getElementById atau querySelector("#judul").
2. **B. elemen PERTAMA ber-class kartu** — querySelectorAll untuk semuanya.
3. **B. btn.addEventListener("click", fn)** — addEventListener adalah cara standar.
4. **B. el.textContent="Hai"** — textContent (aman) atau innerHTML (bisa tag).
5. **B. innerHTML merender tag HTML; textContent teks polos** — Hati-hati XSS bila innerHTML diisi input pengguna.
6. **B. el.classList.add("aktif")** — classList: add, remove, toggle, contains.
7. **B. inp.value** — Elemen input memakai .value.
8. **A. document.createElement("div")** — createElement lalu appendChild/append.
9. **B. mencegah aksi bawaan (mis. form reload)** — Contoh klasik: submit form tanpa reload.
10. **B. input** — Event "input" menyala setiap isi berubah.
11. **document** — document = pintu masuk DOM.
12. **toggle** — toggle cocok untuk dark mode, menu, dll.

Asinkron: Promise, Fetch & Storage

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

JS tidak menunggu operasi lambat — ia memakai **Promise**:

```
async function ambilData() {
  try {
    const res = await
fetch("https://api.example.com/data");
    const data = await res.json();
  } catch (e) { console.error(e); }
}
setTimeout(fn, 2000);   // sekali, 2 dtk
setInterval(fn, 1000);  // berulang
```

Menyimpan data di browser: `localStorage.setItem("skor", JSON.stringify(obj))` lalu `JSON.parse(localStorage.getItem("skor"))` — semua tersimpan sebagai string. Persis teknik yang dipakai KodeQuiz menyimpan progresmu!

🧠 Uji Diri — 10 soal

1. **Promise digunakan untuk...**

   A. loop
   B. menangani operasi asinkron
   C. styling
   D. enkripsi

2. **setTimeout(fn, 2000) akan...**

   A. ulang tiap 2 dtk
   B. jalankan fn SEKALI setelah 2 detik
   C. hentikan fn
   D. 2000 kali

3. **await hanya boleh dipakai dalam fungsi...**

   A. arrow
   B. async
   C. global selalu
   D. callback

4. **fetch("data.json") mengembalikan...**

   A. data langsung
   B. Promise berisi Response
   C. string
   D. XML

5. **JSON.parse() berfungsi...**

   A. objek→string
   B. string JSON→objek JS
   C. validasi
   D. kirim data

6. **localStorage.setItem("skor", 90) menyimpan bertipe...**

   A. number
   B. string
   C. object
   D. boolean

7. **Menangkap error di async/await memakai...**

   A. catch saja di mana pun
   B. try...catch
   C. onerror
   D. if error

8. **[...arr1, ...arr2] menghasilkan...**

   A. array bersarang
   B. gabungan kedua array
   C. error
   D. objek

9. **Kata kunci sebelum function agar bisa await: ____ function ambil()**

   Jawab: _____

10. **Mengubah objek jadi string JSON: JSON.____(obj)**

    Jawab: _____

Kunci & Pembahasan · JavaScript Bab 8

← Kembali ke soal · Awal bab

1. **B. menangani operasi asinkron** — Nilai yang tersedia nanti: pending → fulfilled/rejected.
2. **B. jalankan fn SEKALI setelah 2 detik** — setInterval untuk berulang.
3. **B. async** — async function memungkinkan await.
4. **B. Promise berisi Response** — Perlu await res.json() untuk isinya.
5. **B. string JSON→objek JS** — parse: baca; stringify: simpan/kirim.
6. **B. string** — localStorage selalu string — objek perlu JSON.stringify.
7. **B. try...catch** — try { await ... } catch (e) { ... }
8. **B. gabungan kedua array** — Spread menggabungkan/menyalin array.
9. **async** — async menjadikan fungsi mengembalikan Promise.
10. **stringify** — stringify untuk menyimpan/mengirim data.

JavaScript Modern (ES6+) yang Dipakai Setiap Hari

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Kode JavaScript di perusahaan (React, Vue, Odoo OWL, Node.js) penuh dengan sintaks modern ini:

```
// spread & rest
const b = [...a, 3];                    // salin + tambah
const q = { ...p, kota: "Bandung" };    // salin objek + timpa
function total(...angka) {}             // kumpulkan argumen

// destructuring + default
const { nama, kota = "Tidak ada" } = user;
const [pertama, kedua] = daftar;

// aman membaca data yang mungkin kosong
user.profil?.nama          // undefined, bukan error
diskon ?? 0                // hanya ganti null/undefined (0 tetap 0)

// rantai method array (WHERE → SELECT → gabung)
data.filter(d => d.t >= 5).map(d => d.s).join(", ")

// GROUP BY dengan reduce
const per = trx.reduce((acc, [s, t]) => {
  acc[s] = (acc[s] || 0) + t;
  return acc;
}, {});
```

Method	Hasil	Padanan
`map`	array baru, panjang sama	kolom hitung

`filter`	array lebih pendek	WHERE
`reduce`	satu nilai/objek	SUM / GROUP BY
`find` / `findIndex`	elemen/posisi pertama	XLOOKUP
`some` / `every`	true/false	validasi
`sort((a,b) => a - b)`	urut angka	ORDER BY

> ⚠️ `sort()` tanpa pembanding mengurutkan sebagai teks: `[10, 9, 1, 100] → [1, 10, 100, 9]`.

🧠 Uji Diri — 16 soal

1. **Output kode ini:**

```
const a = [1, 2];
const b = [...a, 3];
console.log(b.length);
```

A. 3
B. 2
C. [1,2,3]
D. 1

2. **Output kode ini:**

```
const p = { nama: "Andi", kota: "Jakarta" };
const q = { ...p, kota: "Bandung" };
console.log(p.kota, q.kota);
```

A. Jakarta Bandung
B. Bandung Bandung
C. Jakarta Jakarta
D. undefined Bandung

3. **Output kode ini:**

```
const user = { profil: null };
console.log(user.profil?.nama);
```

A. undefined
B. null
C. Error: Cannot read properties of null
D. ""

4. **Output kode ini:**

```
console.log(0 ?? 10, 0 || 10);
```

A. 0 10
B. 10 10
C. 0 0
D. 10 0

5. **Output kode ini:**

```
const total = [14, 1.5, 6].reduce((acc, x) =>
acc + x, 0);
console.log(total);
```

A. 21.5
B. 1461.5
C. 3
D. 14

6. **Output kode ini:**

```
const data = [
  { s: "Andi", t: 14 },
  { s: "Bima", t: 1.5 },
  { s: "Citra", t: 6 },
];
console.log(data.filter(d => d.t >= 5).map(d =>
d.s).join(", "));
```

A. Andi, Citra
B. Andi
C. Andi, Bima, Citra
D. 14, 6

7. **Output kode ini:**

```
console.log([10, 9, 1, 100].sort().join(","));
```

A. 1,10,100,9
B. 1,9,10,100
C. 100,10,9,1
D. 9,1,10,100

8. **Output kode ini:**

```
const { nama, kota = "Tidak ada" } = { nama:
"Lia" };
console.log(kota);
```

A. Tidak ada
B. undefined
C. Lia
D. Error

9. **Output kode ini:**

```
function total(...angka) {
  return angka.length;
}
console.log(total(4, 8, 15));
```

A. 3
B. 27
C. 1
D. undefined

10. **Output kode ini:**

```
function buatCounter() {
  let n = 0;
  return () => ++n;
}
const c = buatCounter();
c(); c();
console.log(c());
```

A. 3
B. 1
C. 0
D. 2

11. **Output kode ini:**

```
const teks = JSON.stringify({ a: 1 });
console.log(typeof teks, JSON.parse('{"x": 5}').x);
```

A. string 5
B. object 5
C. string undefined
D. object undefined

12. **Output kode ini:**

```
console.log([5, 8, 12].find(x => x > 6));
```

A. 8
B. [8, 12]
C. true
D. 1

13. **Output kode ini:**

```
const transaksi = [["Andi", 14], ["Bima", 1.5], ["Andi", 4]];
const per = transaksi.reduce((acc, [s, t]) => {
  acc[s] = (acc[s] || 0) + t;
  return acc;
}, {});
console.log(per.Andi);
```

A. 18
B. 14
C. 4
D. undefined

14. **Output kode ini:**

```
console.log((1234567).toLocaleString("id-ID"));
```

A. 1.234.567
B. 1,234,567
C. 1234567
D. Rp1.234.567

15. **Output kode ini:**

```
console.log([3, 5, 7].every(x => x % 2 === 1),
[3, 5, 8].some(x => x % 2 === 0));
```

A. true true
B. true false
C. false true
D. false false

16. **Method array yang mengubah SETIAP elemen dan menghasilkan array baru dengan panjang yang sama: arr.___(x ⇒ x * 2)**

Jawab: _____

Kunci & Pembahasan · JavaScript Bab 9

← Kembali ke soal · Awal bab

1. **A. 3** — Spread (...) 'membuka' isi array ke dalam array baru: b = [1, 2, 3]. Array a tidak berubah. Dipakai untuk menyalin dan menggabungkan array tanpa mengubah aslinya.
   - **B:** b berisi tiga elemen.
   - **C:** Yang dicetak adalah panjangnya, bukan isinya.
   - **D:** Spread menyalin semua elemen.

2. **A. Jakarta Bandung** — Spread objek menyalin semua properti p ke objek BARU, lalu kota ditimpa. p tetap utuh. Pola ini dipakai di React/OWL untuk memperbarui state tanpa mengubah objek lama.
   - **B:** p tidak ikut berubah karena q adalah objek baru.
   - **C:** Properti yang ditulis setelah spread menimpa nilainya.
   - **D:** p.kota masih ada.

3. **A. undefined** — Optional chaining (?.) berhenti dengan aman dan mengembalikan undefined jika bagian sebelumnya null/undefined. Tanpa ?., kode ini error. Sangat berguna saat membaca data JSON dari API yang tidak selalu lengkap.
   - **B:** ?. mengembalikan undefined, bukan meneruskan null.
   - **C:** Error itu terjadi jika ditulis user.profil.nama tanpa ?.
   - **D:** Tidak ada teks kosong di sini.

4. **A. 0 10** — ?? (nullish coalescing) hanya mengganti null/undefined. || mengganti SEMUA nilai falsy (0, "", false, null, undefined). Untuk nilai default angka, pakai ?? supaya 0 tidak ikut terganti — misalnya diskon 0 yang sah.
   - **B:** ?? tidak mengganti 0 karena 0 bukan null/undefined.
   - **C:** || mengganti 0 karena 0 falsy.
   - **D:** Terbalik.

5. **A. 21.5** — reduce memproses array menjadi satu nilai. acc (akumulator) dimulai dari 0, lalu ditambah tiap elemen: 0 → 14 → 15.5 → 21.5.
   - **B:** Itu terjadi jika nilai awalnya string "" (penggabungan teks).
   - **C:** 3 adalah panjang array.
   - **D:** Itu hanya elemen pertama.

6. **A. Andi, Citra** — Rantai method: filter (WHERE t >= 5) → map (SELECT s) → join (gabung jadi teks). Pola filter-map-join ini dipakai untuk membuat isi tabel/daftar di dashboard, seperti di proyek SalesKu.
   - **B:** Citra (6) juga memenuhi syarat ≥ 5.
   - **C:** Bima (1,5) tidak lolos filter.
   - **D:** map mengambil nama (s), bukan angka (t).

7. **A. 1,10,100,9** — Jebakan terkenal: sort() tanpa fungsi pembanding mengurutkan sebagai TEKS ("10" < "9" karena '1' < '9'). Untuk angka tulis sort((a, b) ⇒ a - b). Untuk menurun: (a, b) ⇒ b - a.
   - **B:** Itu hasil sort((a, b) ⇒ a - b), bukan sort() biasa.
   - **C:** Itu urutan menurun numerik.
   - **D:** Bukan urutan teks maupun angka.

8. **A. Tidak ada** — Destructuring mengambil properti ke variabel. = "Tidak ada" adalah nilai default jika propertinya tidak ada (undefined).
   - **B:** Nilai default dipakai saat properti tidak ada.
   - **C:** Lia adalah nilai nama.
   - **D:** Destructuring dengan default adalah sintaks valid.

9. **A. 3** — Rest parameter (...angka) mengumpulkan semua argumen menjadi array [4, 8, 15]. Panjangnya 3. Bedakan: ... di parameter = rest (mengumpulkan), ... saat memanggil/membuat array = spread (membuka).
   - **B:** Itu jumlah angkanya, sedangkan yang dikembalikan length.
   - **C:** Ada tiga argumen.
   - **D:** Fungsi mengembalikan nilai (return).

10. **A. 3** — Closure: fungsi yang dikembalikan 'mengingat' variabel n dari tempat ia dibuat, meskipun buatCounter sudah selesai. Tiga kali dipanggil → 3. Closure adalah dasar event handler dan modul di JavaScript.
    - **B:** n tidak di-reset karena buatCounter hanya dipanggil sekali.
    - **C:** ++n menaikkan nilai sebelum dikembalikan.
    - **D:** Panggilan ketiga mengembalikan 3.

11. **A. string 5** — JSON.stringify: objek → teks JSON (untuk dikirim ke server atau disimpan di localStorage). JSON.parse: teks JSON → objek. Proyek SalesKu memakai keduanya untuk menyimpan data.
    - **B:** stringify menghasilkan string.
    - **C:** parse mengembalikan objek yang punya properti x = 5.
    - **D:** Keduanya keliru.

12. **A. 8** — find mengembalikan elemen PERTAMA yang memenuhi syarat. filter mengembalikan SEMUA dalam array. findIndex mengembalikan posisinya.
    - **B:** Itu hasil filter.
    - **C:** find mengembalikan elemennya, bukan boolean (itu some).
    - **D:** Itu hasil findIndex.

13. **A. 18** — Ini GROUP BY versi JavaScript: reduce dengan akumulator objek, menjumlah t per nama sales. Andi: 14 + 4 = 18. Pola ini persis yang dipakai di langkah ranking proyek SalesKu.
    - **B:** Transaksi Andi yang kedua juga dijumlahkan.
    - **C:** Itu hanya transaksi terakhir Andi.
    - **D:** per.Andi terisi karena ada transaksi Andi.

14. **A. 1.234.567** — toLocaleString dengan locale id-ID memakai titik sebagai pemisah ribuan. Untuk format rupiah lengkap: new Intl.NumberFormat("id-ID", { style: "currency", currency: "IDR" }).
    - **B:** Itu format en-US.
    - **C:** toLocaleString menambahkan pemisah ribuan.
    - **D:** Simbol Rp hanya muncul dengan opsi style currency.

15. **A. true true** — every = apakah SEMUA memenuhi syarat (semua ganjil → true). some = apakah MINIMAL SATU memenuhi (ada 8 yang genap → true). Berguna untuk validasi form.
    - **B:** some menemukan 8 yang genap.
    - **C:** Semua elemen [3, 5, 7] ganjil.
    - **D:** Keduanya bernilai true.

16. **map** — map → array baru dengan panjang sama. filter → array lebih pendek. reduce → satu nilai. forEach → tidak mengembalikan apa-apa.

Latihan Membaca Kode & Jebakan JavaScript

Lompat ke: 🧠 Uji Diri · 🔑 Kunci & Pembahasan

Pertanyaan "apa output kode ini?" adalah favorit interview frontend. Kebanyakan berasal dari sedikit jebakan yang sama:

Kode	Hasil	Kenapa
`"5" == 5`	true	== mengonversi tipe → pakai ===
`3 + 4 + "5"`	"75"	bertemu string → gabung teks
`0.1 + 0.2 === 0.3`	false	desimal biner tidak presisi
`typeof null`	"object"	bug sejarah
`[1,2] === [1,2]`	false	dibandingkan referensinya
`console.log(x); var x = 5`	undefined	hoisting var
`Boolean("0")`, `Boolean([])`	true	bukan falsy

Event loop — urutan eksekusi:

```
1. Kode sinkron (baris demi baris)
2. Microtask: Promise.then, await
3. Macrotask: setTimeout, event klik

console.log("A"); setTimeout(…"B"); Promise…
then(…"C"); console.log("D");
→ A, D, C, B
```

💡 Latih di console browser (F12) atau Node.js: tebak dulu, baru jalankan. Selalu pakai `let` / `const` dan `===` untuk menghindari separuh jebakan di atas.

🧠 Uji Diri — 14 soal

1. **Output kode ini:**

```
console.log(typeof null);
```

A. object
B. null
C. undefined
D. boolean

2. **Output kode ini:**

```
console.log(0.1 + 0.2 === 0.3);
```

A. false
B. true
C. 0.3
D. Error

3. **Output kode ini:**

```
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 0);
}
```

A. 3 3 3
B. 0 1 2
C. 0 0 0
D. Error

4. **Urutan output kode ini:**

```
console.log("A");
setTimeout(() => console.log("B"), 0);
Promise.resolve().then(() => console.log("C"));
console.log("D");
```

A. A, D, C, B
B. A, B, C, D
C. A, D, B, C
D. A, C, D, B

5. **Output kode ini:**

```
console.log("5" == 5, "5" === 5);
```

A. true false
B. true true
C. false false
D. false true

6. **Output kode ini:**

```
console.log([1, 2] === [1, 2]);
```

A. false
B. true
C. undefined
D. Error

7. **Output kode ini:**

```
const o = { n: 1 };
o.n = 2;
console.log(o.n);
```

A. 2
B. 1
C. Error: Assignment to constant variable
D. undefined

8. **Output kode ini:**

```
console.log(x);
var x = 5;
```

A. undefined
B. 5
C. Error: x is not defined
D. null

9. **Output kode ini:**

```
console.log(3 + 4 + "5");
```

A. 75
B. 345
C. 12
D. "12"

10. **Output kode ini:**

```
const arr = [1, 2, 3];
arr[5] = 6;
console.log(arr.length);
```

A. 6
B. 4
C. 3
D. 5

11. **Output kode ini:**

```
async function ambil() {
  return 42;
}
ambil().then(v => console.log(v + 1));
```

A. 43
B. 42
C. Promise
D. undefined

12. **Output kode ini:**

```
console.log(["1", "2",
"3"].map(parseInt).join(","));
```

A. 1,NaN,NaN
B. 1,2,3
C. NaN,NaN,NaN
D. 1,2,NaN

13. **Output kode ini:**

```
console.log(Boolean(""), Boolean("0"),
Boolean([]));
```

A. false true true
B. false false false
C. true true true
D. false false true

14. **Operator perbandingan yang memeriksa nilai DAN tipe data sekaligus (tiga karakter): ___**

Jawab: _____

Kunci & Pembahasan · JavaScript Bab 10

← Kembali ke soal · Awal bab

1. **A. object** — Ini 'bug sejarah' JavaScript sejak versi pertama yang dipertahankan demi kompatibilitas. Cara mengecek null yang benar: nilai === null.
   - **B:** Tidak ada tipe 'null' di hasil typeof.
   - **C:** typeof undefined yang menghasilkan 'undefined'.
   - **D:** null bukan boolean.

2. **A. false** — Angka desimal disimpan dalam biner sehingga 0.1 + 0.2 = 0.30000000000000004. Ini juga terjadi di Python dan Java. Untuk uang: hitung dalam satuan terkecil (rupiah bulat/sen) atau bulatkan sebelum membandingkan.
   - **B:** Hasilnya bukan tepat 0.3.
   - **C:** Yang dicetak hasil perbandingan (boolean).
   - **D:** Kode ini valid.

3. **A. 3 3 3** — var punya satu variabel i untuk seluruh loop. Saat setTimeout berjalan (setelah loop selesai), i sudah 3. Dengan let, setiap putaran punya i sendiri → 0, 1, 2. Alasan utama selalu memakai let/const.
   - **B:** Itu hasil jika memakai let.
   - **C:** i bertambah sampai 3 sebelum callback berjalan.
   - **D:** Kode ini valid.

4. **A. A, D, C, B** — Kode sinkron jalan dulu (A, D). Lalu microtask (Promise.then → C), baru macrotask (setTimeout → B), walau delay-nya 0. Ini pertanyaan event loop yang sangat sering muncul di interview frontend.
   - **B:** setTimeout dan Promise tidak dijalankan langsung.
   - **C:** Promise (microtask) didahulukan daripada setTimeout (macrotask).
   - **D:** D adalah kode sinkron, jadi dicetak sebelum C.
5. **A. true false** — == mengonversi tipe dulu ("5" menjadi 5) sehingga true. === membandingkan nilai DAN tipe, jadi false. Biasakan selalu memakai === agar tidak ada kejutan.
   - **B:** === tidak mengonversi tipe.
   - **C:** == mengonversi string ke angka.
   - **D:** Terbalik.
6. **A. false** — Array dan objek dibandingkan berdasarkan REFERENSI (alamat di memori), bukan isinya. Dua array terpisah selalu berbeda walau isinya sama. Membandingkan isi: JSON.stringify(a) === JSON.stringify(b) untuk kasus sederhana.
   - **B:** Isinya sama, tapi objeknya berbeda.
   - **C:** Hasil perbandingan selalu boolean.
   - **D:** Kode ini valid.

7. **A. 2** — const melarang mengganti VARIABELNYA (o = {...} tidak boleh), tapi isi objeknya tetap boleh diubah. Untuk membekukan isinya: Object.freeze(o).
    - **B:** Isi objek sudah diubah menjadi 2.
    - **C:** Error itu muncul jika menulis o = { n: 2 }.
    - **D:** Properti n ada.

8. **A. undefined** — Hoisting: deklarasi var 'diangkat' ke atas, tapi nilainya belum diisi → undefined. Dengan let/const, mengakses sebelum deklarasi menghasilkan ReferenceError (temporal dead zone) — lebih aman karena kesalahan langsung terlihat.
    - **B:** Nilai 5 baru diisi di baris berikutnya.
    - **C:** Itu terjadi untuk let/const atau variabel yang tidak pernah dideklarasikan.
    - **D:** undefined berbeda dengan null.

9. **A. 75** — Dikerjakan dari kiri: 3 + 4 = 7 (angka), lalu 7 + "5" = "75" (begitu bertemu string, + menjadi penggabungan teks). Sebaliknya "3" + 4 + 5 = "345". Pastikan input form diubah ke angka dengan Number() sebelum dijumlah.
    - **B:** Itu hasil "3" + 4 + 5.
    - **C:** "5" adalah string, jadi tidak dijumlah sebagai angka.
    - **D:** Tidak ada penjumlahan 3 + 4 + 5 sebagai angka.

10. **A. 6** — Mengisi indeks 5 membuat array 'berlubang': indeks 3 dan 4 kosong, dan length menjadi indeks terakhir + 1 = 6. Untuk menambah di akhir, pakai push().
    - **B:** length mengikuti indeks terbesar + 1.
    - **C:** Panjang awal 3, tapi sudah berubah.
    - **D:** Indeks dimulai dari 0, jadi indeks 5 berarti panjang 6.
11. **A. 43** — Fungsi async selalu mengembalikan Promise. Nilai 42 diambil dengan .then (atau await), lalu ditambah 1 → 43.
    - **B:** Nilainya ditambah 1 di dalam then.
    - **C:** then menerima nilai hasil Promise, bukan Promise-nya.
    - **D:** Fungsi mengembalikan 42 yang dibungkus Promise.
12. **A. 1,NaN,NaN** — map mengirim 2 argumen ke parseInt: (nilai, indeks). parseInt("2", 1) dan parseInt("3", 2) memakai basis bilangan yang tidak valid → NaN. Solusi: .map(Number) atau .map(s ⇒ parseInt(s, 10)).
    - **B:** Itu hasil .map(Number).
    - **C:** Elemen pertama berhasil karena basis 0 dianggap basis 10.
    - **D:** "2" dengan basis 1 tidak valid.

13. **A. false true true** — Nilai falsy hanya: false, 0, "" (string kosong), null, undefined, dan NaN. String "0" dan array kosong [] adalah TRUTHY. Akibatnya if (data.length) lebih aman daripada if (data) untuk mengecek array kosong.
    - **B:** "0" dan [] bernilai truthy.
    - **C:** String kosong bernilai falsy.
    - **D:** "0" adalah string yang tidak kosong, jadi truthy.
14. **===** — === (strict equality). Pasangannya !== untuk 'tidak sama'. Hindari == dan != agar tidak ada konversi tipe tersembunyi.